Spiri

SpiriSynq

Release 0.1

Spiri Robotics

Nov 05, 2026

CONTENTS

1 SpiriSynq **3**
- 1.1 Installation . . . 3
- 1.2 How it works . . . 3
- 1.3 Authoritative vs mirror . . . 4
- 1.4 Remote methods . . . 4
- 1.5 Discovery . . . 5
- 1.6 Cross-language compatibility . . . 5
- 1.7 Key configuration options . . . 5
- 1.8 Docs . . . 5
- 1.9 Development . . . 5
- 1.10 Releasing . . . 5
- 1.11 First-time GitHub setup . . . 6
- 1.12 Template . . . 6
- 1.13 License . . . 6

2 Getting Started **7**
- 2.1 Installation . . . 7
- 2.2 Your first syncable object . . . 7
- 2.3 Authoritative vs mirror . . . 9
- 2.4 Topics . . . 9
- 2.5 Discovering objects . . . 10
- 2.6 Remote methods . . . 10
- 2.7 Cleanup . . . 12
- 2.8 Integrating with existing code . . . 12

3 Concepts **15**
- 3.1 Authoritative and mirror nodes . . . 15
- 3.2 How field sync works . . . 15
- 3.3 Type registry . . . 18
- 3.4 Remote methods . . . 18
- 3.5 Lifecycle and garbage collection . . . 23
- 3.6 Publish and receive flags . . . 23
- 3.7 Codecs . . . 24
- 3.8 Per-field QoS . . . 25
- 3.9 Custom publish and receive . . . 27

3.10 Multiple sessions . . . 28

4 CLI Reference **29**
4.1 topic . . . 29
4.2 zenoh . . . 31
4.3 meta . . . 31
4.4 Tab completion . . . 32
4.5 Recipes . . . 32

5 Protocol Specification **35**
5.1 Roles and what each must implement . . . 35
5.2 1. Key space . . . 36
5.3 2. Payload encoding . . . 36
5.4 3. Field updates . . . 37
5.5 4. Tombstones . . . 38
5.6 5. Echo suppression and sequence numbers . . . 38
5.7 6. Built-in queryables . . . 38
5.8 7. RPC methods . . . 40
5.9 8. Discovery . . . 41
5.10 9. Object schema . . . 42
5.11 Implementer's checklist . . . 47

6 API Reference **49**
6.1 Session . . . 49
6.2 Schema . . . 53
6.3 Remote Callables . . . 53
6.4 Syncable Objects . . . 56
6.5 Link . . . 61
6.6 Example Types . . . 62

7 Revision history **71**
7.1 Issuing a revision . . . 71

Python Module Index **73**

Index **75**

Document control

Document number	**SPIRI-SPIRISYNQ-001**
Revision	**0.1**
Status	DRAFT – not for operational use
Effective date	2026-01-01
Issued by	Spiri Robotics

Uncontrolled when printed.

Keeps Python dataclass instances in sync across processes, machines and languages over Zenoh

CHAPTER

ONE

SPIRISYNQ

SpiriSynq keeps Python dataclass instances in sync across processes, machines, and languages over a Zenoh pub/sub network. Define a typed dataclass, point multiple instances at the same topic, and field changes flow between them automatically.

```python
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject

@dataclass
class Telemetry(SyncableObject):
    altitude: float = 0.0
    battery: int = 100

# Process A — owns the data
t = Telemetry("drone/telemetry", synq_authoritive=True)
t.altitude = 42.5  # published to the network automatically

# Process B — mirrors it
mirror = Telemetry.from_topic("drone/telemetry")
print(mirror.altitude)  # 42.5
```

1.1 Installation

```
pip install SpiriSynq
```

Requires Python 3.13+. A Zenoh router is optional for local use — peers discover each other directly.

1.2 How it works

Each syncable field maps to a Zenoh key expression `<topic>/<field>`. When a field changes, psygnal fires a signal, SpiriSynq serialises the new value to YAML, and publishes it. All subscribers on the same topic receive the update and apply it in-place without echo-looping it back.

Nested dataclass fields sync at the sub-field level: a change to `robot.gps.latitude` publishes to `<topic>/gps/latitude`, not `<topic>/gps`. Inherit from `SubSyncableDataclass` to make a nested type fully

evented without Zenoh overhead; alternatively, use `@dataclass(frozen=True)` for immutable value objects that are replaced atomically.

1.3 Authoritative vs mirror

Sync is **bidirectional** — every instance both publishes its own changes and receives changes from others. `synq_authoritive` controls only the queryable side:

- **Authoritative** (`synq_authoritive=True`): registers Zenoh queryables for full-state rehydration on connect, `@remote_method` RPCs, and schema/discovery endpoints. Sets the base topic prefix from the hostname.
- **Mirror** (default): publishes and receives field changes just like an authoritative instance, but forwards RPC calls to the authoritative node rather than running them locally.

```
# Both sides can write — sync is bidirectional
counter = Counter("myapp/counter", synq_authoritive=True)
mirror = Counter.from_topic("myapp/counter")
mirror.value = 99  # received by counter
```

1.4 Remote methods

`@remote_method` exposes a method as a Zenoh queryable. Mirrors call it transparently — the call is routed to the authoritative node and the return value is sent back.

```
from SpiriSynq.remote_callables import remote_method

@dataclass
class Robot(SyncableObject):
    status: str = "idle"

    @remote_method
    def arm(self, mode: str = "auto") -> str:
        self.status = "armed"
        return f"armed in {mode} mode"

# From a mirror — works exactly like a local call
robot = Robot.from_topic("fleet/robot1")
result = robot.arm(mode="manual")  # "armed in manual mode"
```

Generator and async generator methods are supported. The mirror receives a regular Python generator that streams values as each reply arrives.

Custom timeouts: `robot.arm.timeout(5.0)(mode="manual")`.

1.5 Discovery

```python
from SpiriSynq.session import current_session

session = current_session.get()
for metadata in session.list_topics():
    print(metadata["topic"], metadata["classes"])

# Filter by type
for metadata in session.list_topics(type_filter="Robot"):
    robot = Robot.from_topic(metadata["topic"])
```

1.6 Cross-language compatibility

The wire format is plain Zenoh with YAML payloads. Any node that implements the *SpiriSynq protocol* — three built-in callables, per-field puts, and a JSON Schema describing each object — is a first-class participant. No library required on the other end.

1.7 Key configuration options

Field	Default	Purpose
`synq_authoritive`	`False`	Register queryables; answer rehydration and RPC calls
`synq_publish`	`True`	Publish local changes to the network
`synq_receive`	`True`	Apply incoming changes from the network
`sync_lazy_publish`	`False`	Skip publishing when no subscribers are present
`synq_auto_start`	`True`	Call `sync()` automatically on construction

1.8 Docs

The full documentation, including the API reference, is at https://spiri-robotics.github.io/SpiriSynq/. Each release also attaches it as a PDF.

1.9 Development

```
uv sync          # creates .venv and uv.lock; commit uv.lock
make check       # what CI runs: lint, tests, docs
```

1.10 Releasing

1. Move the `## Unreleased` entries in `CHANGES.md` under the new version.
2. Bump `version` in `pyproject.toml`, then `uv lock`.

3. Tag `vX.Y.Z` and push the tag. CI then:
    - publishes the package to PyPI (trusted publishing; the tag must match the version);
    - publishes the docs for that version, with its PDF, beside the others on GitHub Pages;
    - attaches the docs' controlled PDF to the release.

1.11 First-time GitHub setup

Docs are published to GitHub Pages by `.github/workflows/docs.yml`, every version side by side on the `gh-pages` branch. Once the first run on `main` has created that branch, turn Pages on in **Settings → Pages → Build and deployment → Source: Deploy from a branch → `gh-pages`, `/ (root)`**.

1.12 Template

Generated from SpiriProjectTemplates. Pull in template changes with `uvx copier update`.

1.13 License

Licensed under the MIT License.

CHAPTER

TWO

GETTING STARTED

2.1 Installation

```
pip install SpiriSynq
```

SpiriSynq requires Python 3.13+. A Zenoh router is optional for local use — peers can discover each other directly.

2.2 Your first syncable object

Define a dataclass that inherits from `SyncableObject`. Every field becomes a syncable field — changes are published automatically.

Save this as `counter.py` and run it:

```python
import time
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject
from SpiriSynq.remote_callables import remote_method

@dataclass
class Counter(SyncableObject):
    value: int = 0

    @remote_method()
    def reset(self, to: int = 0) -> int:
        self.value = to
        return self.value

counter = Counter("myapp/counter", synq_authoritive=True)
print(f"Publishing on: {counter.synq_absolute_path}")

while True:
    counter.value += 1
    time.sleep(1)
```

```
$ python counter.py
Publishing on: myhost/myapp/counter
```

While it runs, open a second terminal and watch the field changes arrive in real time:

```
$ python -m SpiriSynq.cli topic watch myhost/myapp/counter
received: 2026-01-01T00:00:01.000000+00:00
value: 0
---
received: 2026-01-01T00:00:02.000000+00:00
value: 1
---
received: 2026-01-01T00:00:03.000000+00:00
value: 2
---
```

Or fetch the current state at any point without waiting for an update:

```
$ python -m SpiriSynq.cli topic rehydrate myhost/myapp/counter
!Counter
value: 7
```

To call the `reset` RPC from another process, create a mirror and call the method directly. Save this as `reset_counter.py`:

```python
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject
from SpiriSynq.remote_callables import remote_method

@dataclass
class Counter(SyncableObject):
    value: int = 0

    @remote_method()
    def reset(self, to: int = 0) -> int:
        self.value = to
        return self.value

mirror = Counter.from_topic("myhost/myapp/counter")
result = mirror.reset(to=0)
print(f"Counter reset, value is now {result}")
```

```
$ python reset_counter.py
Counter reset, value is now 0
```

The call is routed to the authoritative process over Zenoh and the return value is sent back. The `counter.py` script will also receive the field update and continue incrementing from 0.

You can also call RPCs directly from the CLI without writing a script:

```
$ python -m SpiriSynq.cli topic call myhost/myapp/counter/reset to=0
0
```

Arguments are passed as `key=value` pairs where values are YAML literals (`to=0` sends an integer, `name=world` sends a string, `flag=true` sends a boolean).

To see all objects currently on the network:

```
$ python -m SpiriSynq.cli topic list
topic: myhost/myapp/counter
classes:
  - '!Counter'
  - '!SyncableObject'
authoritive_node: abc123...
```

2.3 Authoritative vs mirror

Sync is **bidirectional** — any instance can publish field changes and all others on the same topic receive them. The `synq_authoritive` flag controls something narrower: which instance answers queryable calls like `sr_rehydrate` (full-state fetch on connect) and `@remote_method` RPCs.

- `synq_authoritive=True` — responds to rehydration requests and RPC calls. Also sets the base topic prefix from the hostname automatically.
- Mirror (default, `synq_authoritive=False`) — still publishes and receives field changes, but does not register queryables.

```
# Process A
counter = Counter("myapp/counter", synq_authoritive=True)
counter.value = 1

# Process B — can also write
mirror = Counter.from_topic("myapp/counter")
mirror.value = 2  # published back; process A receives it
```

In practice you'll usually have one authoritative instance that "owns" the initial state and handles RPCs, but the field sync itself flows both ways.

2.4 Topics

The first argument to a `SyncableObject` is the **topic** — a slash-separated Zenoh key expression that identifies this object on the network. By default, authoritative objects are published under `<base_topic>/<topic>`. The base topic comes from, in order: the `SPIRI_SYNQ_BASE_TOPIC` environment variable, the contents of `/etc/spirisynq_base_topic`, then the hostname. Per object, `synq_base_topic` overrides it.

In a container, the kernel hostname belongs to the container, not the machine it runs on. Bind-mount the host's `/etc/hostname` to `/etc/spirisynq_base_topic` so every container on a machine shares that machine's

base topic:

```
volumes:
  - /etc/hostname:/etc/spirisynq_base_topic:ro
```

```
counter = Counter("counter", synq_authoritive=True)
print(counter.synq_absolute_path)  # e.g. "myhost/counter"
```

2.4.1 Relative topics

A topic starting with `./` is relative to this node's base topic, so `./counter` and `myhost/counter` name the same object. This works everywhere a topic is accepted — constructors, `from_topic`, `list_topics(prefix="./")`, and the CLI:

```
counter = Counter("./counter", synq_authoritive=True)  # myhost/counter
mirror = Counter.from_topic("./counter")               # same object, from␣
↪this node
other = Counter.from_topic("otherhost/counter")        # absolute: another node
↪'s object
```

`./` always resolves against the *calling* session's base topic, so another node's objects must be named absolutely. `.` is only allowed as the first chunk and `..` is rejected, so a relative marker can never reach the network as a literal key. Use `session.resolve_topic(topic)` to see what a topic expands to.

2.5 Discovering objects

Use `session.list_topics()` to find all objects currently on the network:

```
from SpiriSynq.session import current_session

session = current_session.get()
for metadata in session.list_topics():
    print(metadata["topic"], metadata["classes"])
```

You can filter by type tag:

```
for metadata in session.list_topics(type_filter="Counter"):
    ...
```

2.6 Remote methods

The `Counter` above already has a `@remote_method` — `reset` is callable from any mirror instance and is transparently routed to the authoritative node over Zenoh:

```
mirror = Counter.from_topic("myhost/myapp/counter")
result = mirror.reset(to=0)
print(result)  # 0
```

Generator methods are also supported — the caller receives a regular Python generator that streams values from the authoritative node:

```
@remote_method()
def count_down(self, from_value: int = 10):
    for i in range(from_value, -1, -1):
        self.value = i
        yield i

for value in mirror.count_down(from_value=5):
    print(value)
```

Generator methods also work from the CLI — each yielded value is printed separated by `---`:

```
$ python -m SpiriSynq.cli topic call myhost/myapp/counter/count_down␣
↪from_value=3
3
---
2
---
1
---
0
```

Use `@method.client()` to post-process the return value on the mirror side only — useful for decoding, unit conversion, or any transform you don't want to run on the server:

```
@remote_method()
def get_reading(self) -> bytes:
    return self._sensor.read_raw()

@get_reading.client()
def get_reading(self, result: bytes) -> float:
    return struct.unpack("f", result)[0]

value = mirror.get_reading()  # float, decoded on the caller
```

See *Concepts — Client-side transforms* for full details including generator and async support.

2.7 Cleanup

Objects undeclare their Zenoh resources when garbage collected. For deterministic cleanup, call `obj.close()` explicitly.

2.8 Integrating with existing code

SpiriSynq does not own the main loop. Zenoh I/O runs on its own background threads, so you can drop a `SyncableObject` into any existing application — an asyncio service, a robotics framework, a game loop — without yielding control.

For a long-running producer, spin up your own thread and write to the object's fields from it.

```
import threading
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject

@dataclass
class Sensor(SyncableObject):
    reading: float = 0.0

sensor = Sensor("myapp/sensor", synq_authoritive=True)

def read_loop():
    while True:
        sensor.reading = hardware.read()

threading.Thread(target=read_loop, daemon=True).start()

# your existing main loop continues here uninterrupted
```

For asyncio, use `.as_async()` on remote method calls so you don't block the event loop:

```
mirror = Sensor.from_topic("myhost/myapp/sensor")

async def main():
    result = await mirror.calibrate.as_async()
```

2.8.1 Reacting to field changes

Instead of polling, connect a callback to the field's signal. The callback runs on Zenoh's background I/O thread whenever a change arrives — whether it came from the network or from a local assignment.

```
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject

@dataclass
```

(continues on next page)

(continued from previous page)

```
class Sensor(SyncableObject):
    reading: float = 0.0

mirror = Sensor.from_topic("myhost/myapp/sensor")

def on_reading(value: float):
    print(f"reading changed to {value}")

mirror.events.reading.connect(on_reading)
```

To react to **any** field change on a single object, connect to the group signal. The callback receives a psygnal `EmissionInfo` that carries the field name and the new value:

```
from psygnal import EmissionInfo

def on_any_change(event: EmissionInfo):
    field = event.path[0].attr
    value = event.args[0]
    print(f"{field} changed to {value}")

mirror.events.connect(on_any_change)
```

To stop receiving notifications, call `disconnect` with the same callable:

```
mirror.events.reading.disconnect(on_reading)
```

Asyncio integration

Callbacks run on the Zenoh background thread, not the asyncio event loop. To hand off to asyncio safely, use a `Queue` or `run_coroutine_threadsafe`:

```
import asyncio

async def main():
    mirror = Sensor.from_topic("myhost/myapp/sensor")
    loop = asyncio.get_running_loop()
    queue: asyncio.Queue[float] = asyncio.Queue()

    mirror.events.reading.connect(
        lambda value: loop.call_soon_threadsafe(queue.put_nowait, value)
    )

    while True:
        value = await queue.get()
        print(f"reading: {value}")
```

`call_soon_threadsafe` is the correct bridge — it schedules the `put_nowait` on the event loop from the

Zenoh thread without blocking it. Keep signal callbacks short and non-blocking; hand heavy work off to the event loop or a worker thread.

2.8.2 Handling bad or unexpected data

Three signals fire when an incoming update can't be applied cleanly. Connecting to them lets you log diagnostics, trigger recovery logic, or surface errors to a monitoring system.

`synq_signal_unknown_path` — the incoming path isn't a field on this class at all. This usually means a publisher is sending a field that doesn't exist on the receiver's schema version. The second argument is the raw `zenoh.Sample` (not decoded).

```
def on_unknown(path: str, sample):
    print(f"received unknown field '{path}' — value ignored")

mirror.synq_signal_unknown_path.connect(on_unknown)
```

`synq_signal_type_mismatch` — the path is valid but the decoded value's type doesn't match the field annotation. Fires when `synq_check_receive_types = True` (the default). Both arguments are the relative path and the decoded object.

```
def on_type_mismatch(path: str, obj):
    print(f"wrong type at '{path}': got {type(obj).__name__}")

mirror.synq_signal_type_mismatch.connect(on_type_mismatch)
```

`synq_signal_missing_parent` — the path and type are both valid, but an intermediate nested object is `None`. For example, a publisher sends `bar/value` while this instance has `bar = None`, so the value can't be applied. Both arguments are the relative path and the decoded object.

```
@dataclass
class Robot(SyncableObject):
    arm: Arm | None = None  # if this is None, arm/position updates will emit␣
↪this signal

def on_missing_parent(path: str, obj):
    print(f"can't apply '{path}': parent is None — consider initialising the␣
↪field first")

mirror.synq_signal_missing_parent.connect(on_missing_parent)
```

All three signals fire on the Zenoh background thread, so the same asyncio bridging rules apply as above.

CHAPTER

THREE

CONCEPTS

3.1 Authoritative and mirror nodes

Field sync is **bidirectional** — every instance, whether authoritative or not, both publishes its own changes and receives changes from others on the same topic. `synq_authoritive` controls only the queryable side:

The authoritative instance:

- Registers Zenoh queryables for `@remote_method` RPCs and the built-in `sr_rehydrate`, `sr_metadata`, and `sr_object_schema` endpoints.
- Sets its base topic prefix automatically from the hostname.

Mirror instances (`synq_authoritive=False`):

- Publish and receive field changes just like an authoritative instance.
- Forward `@remote_method` calls to the authoritative node's queryable instead of running them locally.
- Do not register queryables, so they cannot answer rehydration or discovery requests.

SpiriSynq does not enforce a single-authoritative constraint at the protocol level. Running two authoritative instances on the same topic will cause conflicting queryable registrations and undefined rehydration behaviour.

3.2 How field sync works

Each syncable field maps to a Zenoh key expression `<topic>/<field>`. When you assign a new value to a field on an authoritative object, psygnal fires a change signal, SpiriSynq serialises the new value to YAML, and publishes it to `<topic>/<field>`.

Mirrors have a subscriber on `<topic>/**`. When a put arrives, SpiriSynq:

1. Strips the topic prefix to get the relative field path.
2. Validates it against the class's `valid_sync_paths` (derived from field type annotations).
3. Deserialises the YAML payload.
4. Applies it as a DeepDiff delta — mutating the local instance in-place without firing the change signal again, so the update doesn't echo back.

Echo prevention uses Zenoh `SourceInfo`: each session tags its puts with its own Zenoh session ID, and mirrors discard updates from their own ID.

3.2.1 Nested dataclasses

Nested dataclass fields sync at the field level, not the object level. A change to `robot.gps.latitude` publishes to `<topic>/gps/latitude`, not `<topic>/gps`.

For this to work, the nested type must fire change signals on mutation. There are three ways to satisfy this:

`SubSyncableDataclass` (recommended for mutable nested objects)

Inherit from `SubSyncableDataclass` to get psygnal events, `synq_skip_sync`, and `synq_skip_rehydrate` — the same filtering machinery as `SyncableObject` — without any Zenoh publish/subscribe overhead. Individual sub-field assignments propagate automatically:

```
from dataclasses import dataclass
from SpiriSynq.syncable_objects import SyncableObject, SubSyncableDataclass

@dataclass
class GPS(SubSyncableDataclass):
    latitude: float = 0.0
    longitude: float = 0.0
    altitude: float = 0.0

@dataclass
class Robot(SyncableObject):
    gps: GPS = field(default_factory=GPS)

robot = Robot("fleet/robot1", synq_authoritive=True, gps=GPS())

# Each assignment is a separate publish to <topic>/gps/latitude etc.
robot.gps.latitude = 37.77
robot.gps.longitude = -122.41
```

`@dataclass(frozen=True)` (for value objects)

Frozen dataclasses cannot be mutated in-place, so you replace the whole field. SpiriSynq publishes the new value to `<topic>/gps` as a single YAML put:

```
@dataclass(frozen=True)
class Offset:
    x: float = 0.0
    y: float = 0.0

@dataclass
class Arm(SyncableObject):
    offset: Offset | None = None

arm.offset = Offset(x=1.0, y=2.0)  # publishes the whole Offset
```

Use frozen dataclasses when the object is a small immutable value type and you always want to replace it atomically.

If neither condition holds, SpiriSynq logs a warning at startup. Silence it with `warn_non_evented = False` on the parent `SyncableObject`.

Filtering sub-fields

`SubSyncableDataclass` supports the same `synq_skip_sync` and `synq_skip_rehydrate` class variables as `SyncableObject`. Fields listed in `synq_skip_sync` are excluded from the valid sync paths that the parent computes; fields in `synq_skip_rehydrate` are excluded from `to_yaml()` serialisation.

```
@dataclass
class Sensor(SubSyncableDataclass):
    synq_skip_rehydrate = {"raw_buffer"}
    value: float = 0.0
    raw_buffer: bytes = b""  # never included in rehydration snapshots
```

3.2.2 Evented containers

`EventedList`, `EventedDict`, and `EventedSet` (from `psygnal.containers`) can be used as field types and will synchronise automatically. Any mutation — `append`, `__setitem__`, `add`, `discard`, etc. — causes the **entire container** to be re-serialised and published as a single atomic payload:

```
from psygnal.containers import EventedList, EventedDict, EventedSet
from dataclasses import dataclass, field

@dataclass
class Dashboard(SyncableObject):
    readings: EventedList = field(default_factory=EventedList)
    labels: EventedDict = field(default_factory=EventedDict)
    active_flags: EventedSet = field(default_factory=EventedSet)

dash = Dashboard("ui/dashboard", synq_authoritive=True)
dash.readings.append(42.0)       # publishes !EventedList [42.0]
dash.labels["temp"] = "hot"      # publishes !EventedDict {temp: hot}
dash.active_flags.add("alarm")   # publishes !EventedSet [alarm]
```

On the receive side the container is updated **in-place** so existing references stay valid.

Trade-offs. Because the whole container is sent on every mutation, evented containers are less efficient than field-level sync for large or frequently-mutated collections. Prefer them for small sets of values where atomic consistency matters (e.g. a list of active connections, a dict of UI state). For large streams of data, a `bytes` field with a custom codec is more appropriate.

The wire format is a YAML-tagged sequence or mapping:

```
readings: !EventedList [1.0, 2.0, 3.0]
labels:   !EventedDict {temp: hot, humidity: low}
flags:    !EventedSet  [alarm, warning]
```

3.2.3 Links

A `Link` field refers to another SpiriSynq object by its absolute topic. It holds only the topic: it doesn't keep the target alive or check that it exists. `link.resolve(Camera)` fetches a mirror, the same as `Camera.from_topic(link.topic)`; without a class, the target's type must already be registered on the session.

```
from SpiriSynq.link import Link

@dataclass
class Mount(SyncableObject):
    camera: Link | None = None

cam = Camera("./camera", synq_authoritive=True)
mount = Mount("./mount", synq_authoritive=True, camera=Link.to(cam))
# publishes  camera: !Link myhost/camera
```

Links must be absolute, so `Link("./camera")` raises `ValueError`; a relative topic would name a different object on each node that received it. Every session decodes `!Link` without registering anything.

3.2.4 Fields excluded from sync

The `synq_skip_sync` class variable lists fields that are never published or received. All `synq_*` bookkeeping fields are in this set by default. Add your own fields to exclude them:

```
@dataclass
class MyObject(SyncableObject):
    synq_skip_sync = SyncableObject.synq_skip_sync | {"local_cache"}
    value: int = 0
    local_cache: dict = field(default_factory=dict)
```

`synq_skip_rehydrate` additionally excludes fields from the full-state snapshot returned by `sr_rehydrate` — useful for large binary fields like raw images.

3.3 Type registry

SpiriSynq serialises everything as YAML using ruamel.yaml. Each `SyncableObject` subclass is registered with a YAML tag (`!ClassName` by default, or overridden via `yaml_tag`).

When you call `SyncableObject.from_topic()` or `session.register_type_recursive()`, SpiriSynq walks the class's field type annotations and registers every `SyncableObject` subclass it finds, recursively. You must register a type before you can receive it — calling `from_topic()` handles this for you.

3.4 Remote methods

`@remote_method` turns a regular method into a descriptor that behaves differently depending on which side you're on:

- **Authoritative**: calls the method directly in the same process.

- **Mirror**: serialises arguments to YAML query parameters, sends a Zenoh `get` to `<topic>/<method_name>`, and deserialises the reply.

The call is transparent — `mirror.reset()` works identically whether the instance is authoritative or a mirror. The `Counter` from *Getting Started* already shows this pattern:

```
@dataclass
class Counter(SyncableObject):
    value: int = 0

    @remote_method()
    def reset(self, to: int = 0) -> int:
        self.value = to
        return self.value
```

```
mirror = Counter.from_topic("myhost/myapp/counter")
result = mirror.reset(to=0)  # routed to the authoritative process
```

3.4.1 Sync vs async

`@remote_method` works on regular, `async def`, generator, and async generator functions. The method signature drives the behaviour on both sides:

Function type	Authoritative side	Mirror side (sync call)	Mirror side (`.as_async()`)
`def f()`	called directly	blocks until reply	`await`able coroutine
`async def f()`	awaited in a new event loop	blocks until reply	`await`able coroutine
`def f()` with `yield`	called directly, returns generator	returns generator; blocks per `next()`	async generator
`async def f()` with `yield`	iterated in a new event loop	returns generator; blocks per `next()`	async generator

On the authoritative side, `async def` methods are always run to completion synchronously (via `asyncio.run`) when called from a sync context. On the mirror side, the Zenoh round-trip is always blocking by default — use `.as_async()` to avoid blocking an event loop.

```
# Sync call — blocks until the authoritative node replies
result = mirror.reset(to=0)

# Async call — awaitable, doesn't block the event loop
result = await mirror.reset.as_async(to=0)
```

3.4.2 Generator methods

If the decorated function uses `yield`, the mirror receives a regular Python generator that streams values as each Zenoh reply arrives. Zenoh consolidation is disabled for these calls so no intermediate values are dropped.

```
@dataclass
class Counter(SyncableObject):
    value: int = 0

    @remote_method()
    def count_down(self, from_value: int = 10):
        for i in range(from_value, -1, -1):
            self.value = i
            yield i
        return "done"

# Mirror side — iterates as replies arrive
for value in mirror.count_down(from_value=5):
    print(value)

# Or async
async for value in mirror.count_down.as_async(from_value=5):
    print(value)
```

The return value of a generator method (the value passed to `StopIteration`) is transmitted as the final reply and is accessible as the `StopIteration` value if you iterate manually with `next()`.

Async generators cannot carry a return value — the final reply payload is `None`.

3.4.3 Client-side transforms

`@method.client()` registers a post-processing function that runs on the mirror side after the RPC reply is received. It is never called on the authoritative side.

```
@dataclass
class Camera(SyncableObject):

    @remote_method()
    def capture(self) -> bytes:
        return self._device.read_frame()

    @capture.client()
    def capture(self, result: bytes) -> np.ndarray:
        return decode_jpeg(result)
```

The client function receives `self` (the mirror instance) and the deserialized return value, and returns the transformed result. Having access to `self` lets you use instance state in the transform — a local calibration matrix, a unit preference, etc.

```
mirror = Camera.from_topic("robot/camera")
frame = mirror.capture()  # returns np.ndarray on the caller, bytes never␣
↪leaves the library
```

On the authoritative object, `capture()` returns `bytes` directly and the client function is not invoked.

For generator methods the client function is applied **per yielded item**:

```
@dataclass
class Sensor(SyncableObject):

    @remote_method()
    def stream(self):
        while True:
            yield self._read_raw()

    @stream.client()
    def stream(self, item: bytes) -> float:
        return struct.unpack("f", item)[0]

# Mirror side — each item is already decoded
for value in mirror.stream():
    print(value)  # float
```

Client transforms also apply when using `.as_async()`.

raw=True — take the raw Zenoh reply

Pass `raw=True` to skip deserialization entirely and receive the raw `zenoh.Reply` object instead. The client function is then responsible for extracting and parsing whatever it needs.

```
import zenoh

@dataclass
class Robot(SyncableObject):

    @remote_method()
    def get_state(self) -> Self:
        return self

    @get_state.client(raw=True)
    def get_state(self, reply: zenoh.Reply) -> dict:
        from ruamel.yaml import YAML
        plain = YAML()
        plain.constructor.add_multi_constructor(
            '', lambda loader, _tag, node: loader.construct_mapping(node,␣
↪deep=True)
```

(continues on next page)

(continued from previous page)

```
        )
        return plain.load(reply.ok.payload.to_string())
```

This is useful when the return type is a `SyncableObject` subclass and you want to avoid the full deserialisation cost (which would otherwise start a new Zenoh session), when you need to inspect the reply encoding before deciding how to decode, or when you need to apply the result to `self` in-place rather than returning a new object.

3.4.4 Server-side transforms

`@method.server()` registers a function that intercepts the raw Zenoh query before the default dispatch logic runs. It receives `self` (the authoritative instance) and the raw `zenoh.Query` object, and is fully responsible for calling the underlying method, encoding the result, and sending the reply.

Use `decode_parameters(query)` rather than `query.parameters` to read the arguments: it undoes the percent-encoding clients apply to `%` and `;`.

```
import zenoh
from SpiriSynq.remote_callables import decode_parameters

@dataclass
class Robot(SyncableObject):

    @remote_method()
    def move(self, x: float, y: float) -> bool:
        return self._actuator.go(x, y)

    @move.server()
    def move(self, query: zenoh.Query):
        registry = self.synq_session.type_registry
        params = {k: registry.load(v) for k, v in decode_parameters(query).
↪items()}
        if not self._auth.check(params.get("token")):
            query.reply_err("Unauthorized")
            return
        result = self.move.__wrapped__(self, **{k: v for k, v in params.
↪items() if k != "token"})
        query.reply(f"{self.synq_absolute_path}/move", payload=registry.
↪dumps(result), encoding=zenoh.Encoding.APPLICATION_YAML)
```

Reply on the method's own key rather than `query.key_expr`: the query may be a wildcard (e.g. `**/move`), and replying with it would hide which object the reply came from.

The server function is **only** invoked by the Zenoh callback — direct local calls on an authoritative instance skip it entirely and call the original method directly. This keeps local usage fast and unaffected.

Common uses:

- **Authentication / authorisation** — inspect query parameters or payload before dispatching.

- **Custom encoding** — accept or return binary payloads that the default YAML path cannot handle.
- **Streaming from external sources** — send multiple replies (e.g. from a hardware buffer) without requiring a generator on the Python side.

If the server function raises an exception, SpiriSynq catches it and calls `query.reply_err()` automatically, matching the behaviour of the default path.

3.4.5 Errors

If the authoritative side raises an exception, the mirror receives an `RpcException` with the error message as its string. The original traceback is logged on the authoritative side.

3.4.6 Timeouts

`.timeout(seconds)` returns a bound method with a custom timeout. The default is the Zenoh session default.

```
result = robot.arm.timeout(5.0)(mode="manual")
await robot.arm.timeout(5.0).as_async(mode="manual")
```

3.5 Lifecycle and garbage collection

Zenoh resources (publishers, subscribers, queryables) are tied to the `SyncableObject` instance. When the object is garbage collected, `__del__` calls `close()` to undeclare them. SpiriSynq uses weak references for internal callbacks so that objects are not kept alive by their own Zenoh subscriptions.

The `Session` object is similarly cleaned up on GC, and is also registered with a shutdown hook so that Zenoh's non-daemon threads don't prevent interpreter exit.

3.6 Publish and receive flags

Four boolean fields on `SyncableObject` control whether an instance participates in the pub/sub exchange:

Field	Default	Effect
`synq_publis`	`True`	Publish local field changes to Zenoh. Set to `False` for a receive-only instance.
`synq_receiv`	`True`	Apply incoming changes from Zenoh. Set to `False` for a publish-only instance.
`sync_lazy_p`	`Fals`	Skip publishing when there are no active subscribers. Reduces network traffic at the cost of subscribers that join late potentially missing updates.
`synq_auto_s`	`True`	Call `sync()` automatically in `__post_init__`. Set to `False` if you need to finish configuring the object before it begins synchronising.

`synq_check_receive_types` (default `True`) validates that incoming values match the field's type annotation before applying them. A mismatch logs a warning and discards the update. Disable it only if you are deliberately receiving subtypes or loosely-typed values.

3.7 Codecs

By default, SpiriSynq serialises every field value as YAML before putting it on the Zenoh network. For most types this is fine, but for binary data — images, audio, raw sensor buffers — YAML base64-encodes the bytes, adding roughly 33 % size overhead and a full YAML parse on receive.

Codecs let you register a custom serialiser/deserialiser pair on a `Session` for a specific Python type. When a codec is registered, it takes over from YAML for that type in both directions.

3.7.1 How codec selection works

- **Encoding (publish)**: when a field value is about to be put to Zenoh, SpiriSynq walks the value's MRO and picks the first registered codec whose `python_type` matches. If none matches, it falls back to YAML.
- **Decoding (receive)**: when a sample arrives, SpiriSynq checks the sample's Zenoh encoding against each registered codec's `zenoh_schema`. If one matches, that codec's `decode` method is called instead of the YAML parser.

This means the encoding carried in the Zenoh message is the unambiguous signal for which decoder to use — no out-of-band negotiation required.

3.7.2 Built-in codecs

`BytesCodec` is registered on every `Session` by default. It transmits `bytes` fields as raw `APPLICATION_OCTET_STREAM` payloads, bypassing YAML entirely:

```
@dataclass
class Camera(SyncableObject):
    frame: bytes = b""  # published as raw binary, not base64 YAML
```

No configuration is needed — any `bytes`-typed field is automatically handled.

3.7.3 Writing a custom codec

Subclass `Codec`, set `python_type` and `zenoh_schema` as class attributes, and implement `encode` and `decode`:

```
import numpy as np
import zenoh
from SpiriSynq.codecs import Codec

class JpegCodec(Codec):
    python_type = np.ndarray
    zenoh_schema = zenoh.Encoding.IMAGE_JPEG

    def encode(self, value: np.ndarray) -> tuple[bytes, zenoh.Encoding]:
        ok, buf = cv2.imencode(".jpg", value)
        return buf.tobytes(), zenoh.Encoding.IMAGE_JPEG
```

(continues on next page)

(continued from previous page)

```
    def decode(self, sample: zenoh.Sample) -> np.ndarray:
        buf = np.frombuffer(sample.payload.to_bytes(), dtype=np.uint8)
        return cv2.imdecode(buf, cv2.IMREAD_COLOR)
```

Then register it on the session before any objects are created:

```
from SpiriSynq.session import Session

session = Session()
session.register_codec(JpegCodec())

@dataclass
class Camera(SyncableObject):
    frame: np.ndarray | None = None  # encoded as JPEG, decoded back to ndarray
```

Codecs are checked in registration order; the first match wins. Built-in codecs are prepended, so user-registered codecs take priority over them.

3.7.4 What codecs do not cover

Codecs apply only to the field pub/sub path. They do not affect:

- **RPC parameters and replies** — `@remote_method` arguments and return values are always YAML-serialised.
- **`sr_rehydrate`** — the full-state snapshot uses YAML. If a `bytes` field should be excluded from rehydration snapshots (e.g. a large image), add it to `synq_skip_rehydrate`.

3.8 Per-field QoS

A `SyncableObject` can set Zenoh QoS for one field by declaring a class attribute `<field>_qos: ClassVar[SynqQoS]`:

```
from typing import ClassVar
import zenoh
from SpiriSynq.qos import SynqQoS

@dataclass
class Camera(SyncableObject):
    image: bytes = b""
    image_qos: ClassVar[SynqQoS] = SynqQoS(priority=zenoh.Priority.DATA_HIGH)
```

`SynqQoS` is a frozen dataclass with three fields:

Field	Default	Meaning
`priority`	`zenoh.Priority.DATA`	A `zenoh.Priority`, from `REAL_TIME` (highest) to `BACKGROUND` (lowest).
`congestion_control`	`zenoh.CongestionControl.DROP`	A `zenoh.CongestionControl`.
`express`	`False`	Send without batching.

Plain ints are rejected with a `TypeError`. Use the `zenoh` enum members.

The attribute counts only if it is an instance of `SynqQoS`, so an ordinary field that happens to be called `mode_qos` stays an ordinary field. Because it is a `ClassVar`, it is never synced, never rehydrated, and never appears in the schema. QoS is local to the sender and doesn't travel over the network.

Assigning it on an instance (`obj.image_qos = SynqQoS(...)`) overrides the class default for later publishes and emits no event, which lets a producer change priority from one publish to the next.

`obj.synq_qos_for(path)` returns the resolved QoS for a path, or `None` for Zenoh's defaults. Container publishes (`EventedList`/`EventedDict`/`EventedSet`) use the QoS of their top-level field.

3.8.1 Nested fields

Each `<field>_qos` goes on the class that declares the field, so for a nested dataclass it goes on the nested class, not the root. Resolving `mid/leaf/x` walks the live objects and checks, from the innermost outward:

Segment	Looked up on	Attribute
`x`	`obj.mid.leaf`	`x_qos`
`leaf`	`obj.mid`	`leaf_qos`
`mid`	`obj`	`mid_qos`

The first `SynqQoS` found wins, checking the instance attribute before the class attribute at each level. A QoS on a parent field therefore covers its whole subtree unless a descendant sets its own:

```
@dataclass
class Leaf(SubSyncableDataclass):
    x: int = 0
    y: int = 0
    x_qos: ClassVar[SynqQoS] = SynqQoS(priority=zenoh.Priority.REAL_TIME)

@dataclass
class Mid(SubSyncableDataclass):
    leaf: Leaf | None = None
    z: int = 0
    leaf_qos: ClassVar[SynqQoS] = SynqQoS(priority=zenoh.Priority.
↪INTERACTIVE_HIGH)
```

(continues on next page)

(continued from previous page)

```
@dataclass
class Robot(SyncableObject):
    mid: Mid | None = None
    mid_qos: ClassVar[SynqQoS] = SynqQoS(priority=zenoh.Priority.BACKGROUND)

# mid, mid/z          -> BACKGROUND         (mid_qos)
# mid/leaf, .../y     -> INTERACTIVE_HIGH   (leaf_qos)
# mid/leaf/x          -> REAL_TIME          (x_qos)
```

Some consequences:

- Instance overrides work at any depth: `robot.mid.leaf.y_qos = SynqQoS(...)` affects `mid/leaf/y`. Frozen nested dataclasses can only use class attributes.
- The walk follows the live objects, not the annotations. If a parent is `None`, the nearest live ancestor's QoS applies.
- Replacing a whole sub-object (`robot.mid = Mid(...)`) is one put at `mid` with `mid_qos`. QoS declared inside `Mid` or `Leaf` only applies to in-place changes of their fields.

3.9 Custom publish and receive

An object can take over the wire format of a field by defining `<field>_publish` and/or `<field>_receive`:

```
from typing import Iterator
from SpiriSynq.syncable_objects import SKIP, PutArgs

def image_publish(self, value) -> Iterator[PutArgs | bytes | str]: ...
def image_receive(self, value, sample) -> bytes: ...  # or return SKIP
```

When `_publish` is present, it replaces the default publish for that field. It yields the puts to send, and SpiriSynq sends each one to `<topic>/<field>` with the object's publisher `source_info`, so the echo filter keeps working.

Each yielded dict holds keyword arguments for Zenoh's `Session.put`, so any argument Zenoh's `put` accepts can go in it:

- `payload` is required.
- The optional keys are `encoding`, `priority`, `congestion_control`, `express`, `attachment`, `timestamp` and `allowed_destination`.
- QoS keys you leave out come from the field's resolved `_qos`.
- `key_expr` and `source_info` are set by SpiriSynq; yielding either raises `ValueError`.
- Bare `bytes` or `str` is shorthand for `{"payload": ...}`.

Yielding nothing sends nothing, and returning a list works as well as a generator. Because the hook only describes what to send, you can test it without a network: `list(cam.image_publish(jpeg))`.

`PutArgs` is a `TypedDict`. Annotating the hook as above lets a type checker (mypy, pyright, your editor) flag misspelled keys, a missing `payload`, or wrong value types. At runtime the dict goes straight to Zenoh, so a misspelled key raises Zenoh's own `TypeError` on publish. If a later Zenoh release adds a `put` argument, it works at runtime right away, but a type checker reports it as an unknown key until `PutArgs` is updated.

`_receive` runs on the Zenoh callback thread, inside the receive guard (so applying its result doesn't echo back out), after codec decoding. It returns the value to apply, or `SKIP` to leave the field unchanged. The type check against the field's annotation runs on that returned value, not on the raw sample.

Like `_qos`, hooks go on the class that declares the field. For `mid/leaf/x`, SpiriSynq calls `x_publish` / `x_receive` on `obj.mid.leaf`, with `self` bound to that sub-object. This works for `SubSyncableDataclass` and frozen dataclasses alike. Unlike QoS, hooks have no ancestor fallback: `mid_publish` on the root handles only puts at `mid` (replacing the whole sub-object), never changes to `mid/z`. A receive whose parent is `None` on the receiving side finds no hook and takes the usual missing-parent path.

The two hooks are optional and independent of each other. A peer without them sees whatever the publisher actually sent, so a format with custom framing should be designed so that a hook-less peer fails safely. Hooks only apply to field updates. `sr_rehydrate` still sends the whole value as YAML.

The motivating example is progressive MJPEG: one assignment of a full JPEG becomes several chunks at different priorities, and the receiver reassembles them into a JPEG that refines as chunks arrive:

```
@dataclass
class Camera(SyncableObject):
    image: bytes = b""
    image_qos: ClassVar[SynqQoS] = SynqQoS(priority=zenoh.Priority.DATA_LOW)
    _scans: dict = field(default_factory=dict)  # leading underscore: not␣
↪synced

    def image_publish(self, value) -> Iterator[PutArgs]:
        for i, chunk in enumerate(split_progressive_scans(value)):
            args: PutArgs = {"payload": bytes([i]) + chunk}
            if i == 0:
                # coarse first scan jumps the queue; refinements use image_qos
                args["priority"] = zenoh.Priority.INTERACTIVE_HIGH
            yield args

    def image_receive(self, value, sample):
        self._scans[value[0]] = value[1:]
        if 0 not in self._scans:
            return SKIP
        return assemble_jpeg(self._scans)
```

3.10 Multiple sessions

Most applications use the default session created at import time and never touch `Session` directly. If you need to connect to multiple Zenoh networks in one process, you can create additional `Session` instances and pass them via `synq_session=`, or use `session.as_default()` as a context manager to set the default for a block of code.

CHAPTER

FOUR

CLI REFERENCE

SpiriSynq ships a command-line tool for inspecting and interacting with objects on the network without writing any Python.

```
python -m SpiriSynq.cli [OPTIONS] COMMAND [ARGS]...
```

`--truncate / --no-truncate` — long lines are truncated by default on a TTY and left unwrapped when piped. Override with `--no-truncate`.

Every topic argument and `--prefix` accepts a relative topic: `./camera` means `<base_topic>/camera` for this machine (the hostname, or `SPIRI_SYNQ_BASE_TOPIC`), and `.` alone means this machine's whole tree. No shell quoting is needed.

```
python -m SpiriSynq.cli topic watch './camera/**'
python -m SpiriSynq.cli topic list --prefix .
```

4.1 topic

4.1.1 `topic list`

Discover all objects currently on the network.

```
python -m SpiriSynq.cli topic list
python -m SpiriSynq.cli topic list --type Counter
python -m SpiriSynq.cli topic list --prefix myhost
```

Options:

- `-t / --type` — filter by type name (matches the YAML tag without `!`)
- `-p / --prefix` — restrict the search to a key prefix

4.1.2 `topic watch`

Subscribe to a topic and stream updates to stdout as YAML (default) or NDJSON. Wildcards are supported.

```
python -m SpiriSynq.cli topic watch myhost/drone/telemetry
python -m SpiriSynq.cli topic watch 'myhost/**'
python -m SpiriSynq.cli topic watch myhost/drone/telemetry --json
```

Options:

- `--show-paths / --no-show-paths` — include the key path in each record (default: on when wildcards are used)
- `--json / -j` — emit NDJSON instead of YAML
- `--timestamp / -t` — include the Zenoh message timestamp
- `--received-timestamp / -rt` — include the local receive time (default: on)
- `--count / -n` — exit after receiving N messages (default: 0 = unlimited); useful for scripting or waiting for a topic to appear

Records are separated by `---` when metadata fields are present, making the output valid multi-document YAML and compatible with `topic put`.

Binary payloads are base64-encoded and emitted as a YAML `!!binary` scalar.

4.1.3 `topic put`

Publish a value to a topic. Accepts a direct argument or a piped stream from `topic watch`.

```
# Direct publish
python -m SpiriSynq.cli topic put myhost/drone/telemetry/altitude 42.5

# Replay a captured stream
python -m SpiriSynq.cli topic watch myhost/drone/telemetry | \
    python -m SpiriSynq.cli topic put
```

Options:

- `--input-type auto|yaml|raw` — how to interpret stdin. `auto` tries YAML first and falls back to raw string. `raw` publishes the entire stdin as a single opaque value.

When the piped stream contains `path:` fields (from `topic watch --show-paths`), each record is published to its original path. If a topic prefix is also given, stream paths must be subpaths of it — this prevents accidentally replaying a stream to the wrong part of the tree.

4.1.4 `topic schema`

Fetch and display the field schema for a topic.

```
python -m SpiriSynq.cli topic schema myhost/drone/telemetry
```

4.1.5 `topic rehydrate`

Fetch the full current state of an object. Useful when you suspect local state is stale, or when connecting to a network without a caching router.

```
python -m SpiriSynq.cli topic rehydrate myhost/drone/telemetry
```

4.2 zenoh

Low-level Zenoh diagnostics.

4.2.1 `zenoh info`

Show the local session's ZID and any connected routers and peers.

```
python -m SpiriSynq.cli zenoh info
```

4.2.2 `zenoh scout`

Discover Zenoh routers and peers on the network.

```
python -m SpiriSynq.cli zenoh scout
python -m SpiriSynq.cli zenoh scout --timeout 3.0 --no-peers
```

Options:

- `--timeout / -t` — scouting duration in seconds (default: 1.0)
- `--routers / --no-routers` — include routers (default: on)
- `--peers / --no-peers` — include peers (default: on)
- `--clients / --no-clients` — include clients (default: off)

4.3 meta

4.3.1 `meta type_schema`

Retrieve the schema for a registered type by name. Without a type argument, returns all registered types on the network.

```
python -m SpiriSynq.cli meta type_schema
python -m SpiriSynq.cli meta type_schema Counter
python -m SpiriSynq.cli meta type_schema Counter --prefix myhost
```

Options:

- `-p / --prefix` — restrict the search to a key prefix (default: `**`)

4.4 Tab completion

`synq completion SHELL` prints a completion script for `bash`, `zsh` or `fish`. The bash and zsh scripts cover both `synq` and `spirisynq`.

```
# zsh: ~/.zfunc must be in $fpath before compinit
synq completion zsh > ~/.zfunc/_synq
# bash
synq completion bash > ~/.local/share/bash-completion/completions/synq
```

Regenerate the script after upgrading. It completes:

- topics, one `/` chunk at a time, for every topic argument and `--prefix`. `./` stands for this machine's base topic.
- type tags, for `--type` and `meta type_schema`.
- the method in `topic call` below an object's topic, with its docstring's first line. Names starting with `sr_` only show up once you've typed `sr_`.
- `topic call`'s `key=value` arguments. Keys come from the method's parameters with their types, required ones first, and keys you've already given are left out. Values come from the parameter's type hint: the members of an `Enum` or `Literal`, `true`/`false` for a `bool`, `null` for an optional, and the default.

The first Tab starts a background server, `synq _complete-server`. It keeps a zenoh session open, re-scans `**/sr_metadata/*` every 5 seconds and fetches an object's schema when a Tab needs it. The shell talks to it directly (zsh over a Unix socket, bash over TCP on 127.0.0.1 with a token), so a Tab doesn't wait for Python to start. The server exits after 10 minutes without a Tab. Until it's running, Tabs complete topics and tags from the list it last saved to `~/.cache/spirisynq/topics`, without methods or arguments. Its socket is in `$XDG_RUNTIME_DIR/spirisynq`.

The server only reads metadata and schemas, and never calls a method. It uses the base topic of the shell that asks: `SPIRI_SYNQ_BASE_TOPIC` if that's set, otherwise the hostname.

4.5 Recipes

Capture and replay a topic:

```
python -m SpiriSynq.cli topic watch --no-received-timestamp␣
↪myhost/drone/telemetry > capture.yaml
cat capture.yaml | python -m SpiriSynq.cli topic put myhost/drone/telemetry
```

Monitor all field changes across a host:

```
python -m SpiriSynq.cli topic watch --json 'myhost/**'
```

Check what's on the network:

```
python -m SpiriSynq.cli topic list
python -m SpiriSynq.cli zenoh scout
```

CHAPTER

FIVE

PROTOCOL SPECIFICATION

This page describes what a node must implement to interoperate with SpiriSynq without using this library. Everything here is plain Zenoh (1.x) — there is no SpiriSynq-specific transport, framing or handshake. Every example payload on this page was captured from the reference implementation.

5.1 Roles and what each must implement

A node can take part at three levels. Each level includes the one before it.

Role	Must implement	Sections
Observer — watches an object	Subscribe to `<topic>/**`, decode *payloads*, apply *field updates*, handle *tombstones*	1–4
Mirror — reads and writes an object it doesn't own	Observer, plus fetch initial state with `sr_rehydrate`, publish field puts with a `SourceInfo`, discard its own echoes, call *RPCs*	1–7
Authoritative — owns an object	Mirror, plus implement the three *built-in callables*, declare *RPC queryables*, publish a tombstone on shutdown, describe itself with the *schema*	all

SpiriSynq doesn't decide who may write. Any node that can put to `<path>/<field>` changes the object for everyone, and receivers apply puts in arrival order (last writer wins). Being authoritative only means that the node answers the queryables and owns the object's lifetime.

Permissions are enforced by Zenoh, not by this protocol. Use Zenoh's access control (the `access_control` section of the Zenoh config), configured on the routers, to say which nodes may `put` or `delete` under a key expression, send `query`s (RPC calls) to it, or `declare_queryable` on it. Every SpiriSynq operation is a plain Zenoh message on the object's keys, so those rules cover field writes, tombstones, RPCs and the built-in callables without any SpiriSynq support. Rules only apply to traffic that passes through a node enforcing them, so a deployment that relies on them should route all traffic through those routers rather than letting peers connect to each other directly.

There must be only one authoritative node per topic. Two is undefined behaviour: it won't break immediately, because both nodes answer every query and clients get one of their replies, but the object's state is no longer well defined.

5.2 1. Key space

An object lives at an **absolute path**, a Zenoh key expression:

```
<base_topic>/<topic>          e.g.  myhost/probe
```

- `base_topic` defaults to the authoritative node's hostname (override with `SPIRI_SYNQ_BASE_TOPIC`). It's a naming convention only. On the wire there is just the absolute path.
- The path must be a valid Zenoh key expression **without wildcards**: no empty chunks, no leading or trailing `/`, no `*`, `**` or `$`. The chunks `.` and `..` are also rejected. The Python API uses a leading `./` locally to mean "relative to my base topic", and it's always expanded before anything goes on the wire.
- Under the absolute path, the following chunk names are reserved:

Key	Purpose
`<path>/<field>[/<subfield>…]`	Field values (*§3*)
`<path>/sr_rehydrate`	Full state (*§6*)
`<path>/sr_metadata`, `<path>/sr_metadata/<Tag>`	Discovery metadata (*§6*)
`<path>/sr_object_schema`, `<path>/sr_type_schema/<Tag>`	Object schema (*§6*, *§9*)
`<path>/<method>`	RPC methods (*§7*)

Field names and method names share one namespace below the path, and names starting with `sr_` are reserved for the protocol.

5.3 2. Payload encoding

5.3.1 YAML

Unless a field uses a *codec*, every payload (field puts, RPC arguments, RPC replies and queryable replies) is a **UTF-8 YAML 1.1 document** sent with Zenoh encoding `application/yaml`. The reference implementation uses PyYAML.

- The document end marker (`\n...`) is stripped, so a scalar is sent as just `3.0` rather than `3.0\n...`. Receivers must accept documents both with and without it.
- **Local tags** name application types: `!ClassName` by default, or a class's custom `yaml_tag`. A tagged mapping is an object of that class:

  ```
  !Gimbal
  pitch: 1.0
  yaw: 0.0
  ```

  A receiver that doesn't know a tag must not crash. It can load the node as an untagged mapping (the reference client does this for `sr_rehydrate`), or reject that one update.
- **Standard YAML tags** you will see:

Python value	On the wire
`int`, `float`, `str`, `bool`, `None`	plain scalars (`42`, `3.0`, `hello`, `true`, `null`)
`list`, `tuple`	plain sequence (tuples aren't tagged, so they arrive as sequences)
`dict`, `TypedDict`	plain mapping
`set`	`!!set {5: null}`
`bytes` *(inside a larger document, e.g. rehydrate)*	`!!binary` with base64 text
`EventedList` / `EventedDict` / `EventedSet`	`!EventedList [...]`, `!EventedDict {...}`, `!EventedSet [...]`
`Link` (a reference to another object)	`!Link myhost/camera`, a scalar holding the target's absolute topic

5.3.2 Binary and codec-encoded fields

When a field's value has a registered codec, the field put uses the codec's raw bytes and its Zenoh encoding instead of YAML. The receiver chooses a decoder by matching the **sample's encoding**, not the field type. The only built-in codec is:

Python type	Encoding	Payload
`bytes`	`zenoh/bytes`	raw bytes, unmodified

Applications can register more, such as `image/jpeg`. Codecs apply only to field puts. Inside `sr_rehydrate` and RPC payloads, `bytes` values are still `!!binary` YAML. If a receiver gets an encoding it has no decoder for, it should treat the sample as undecodable and drop it.

5.4 3. Field updates

A change to one field is one Zenoh **put**:

```
key:          <path>/<field>              myhost/probe/speed
encoding:     application/yaml            (or a codec's encoding)
payload:      the field's new value       2.5
source_info:  see §5
```

- **Nested dataclass fields:** setting a nested dataclass field (`obj.gimbal.pitch = 3.0`) publishes at `<path>/gimbal/pitch`. Replacing the whole sub-object publishes the tagged object at `<path>/gimbal`.
- **Containers** (`EventedList`/`EventedDict`/`EventedSet`) are always sent whole, at the container's top-level field key, even after a single `append`. Mutating a plain `list`/`dict`/`set` in place publishes nothing, because only assignment does.
- **Subscribing:** subscribe to `<path>/**`. Remove the `<path>/` prefix to get the relative field path. Ignore any path that isn't a field of the object, and any value whose type doesn't match the field.

- **Missing parents:** an update to `gimbal/pitch` can arrive while `gimbal` is `null` locally. It can't be applied. The reference client drops it and calls `sr_rehydrate` (at most once every 5 s by default).
- **QoS:** priority, congestion control and express are chosen by the publisher for each field. They aren't part of the payload, and receivers need do nothing about them.
- **Custom framing:** an object can replace a field's encoding with its own framing (a publish hook), possibly as several puts per change. A receiver that doesn't understand that framing should treat those samples like any other undecodable update.

5.5 4. Tombstones

When an authoritative object shuts down, it sends a Zenoh **delete** (`SampleKind.DELETE`) on the key expression `<path>/**` (literally, with the wildcard). The delete is sent `RELIABLE` from the object's own publisher, so it carries the same entity ID as the object's field puts and the next `source_sn` in their sequence. A receiver that tracks sequence numbers therefore knows whether it missed any updates before the tombstone. When a receiver sees a delete whose key is `<path>/**` or `<path>`, it should mark the object deleted. Other delete samples under the path should be ignored.

5.6 5. Echo suppression and sequence numbers

Every field put and tombstone carries a Zenoh `SourceInfo`:

- `source_id`: the **entity global ID of the publisher that sent it**. This is the Zenoh session ID (`zid`) plus the publisher's entity ID (`eid`). It isn't just the session.
- `source_sn`: a counter that starts at 0 and goes up by 1 for each sample **from that publisher**. Each object has exactly one publisher and therefore one counter, which all of its field keys and its tombstone share. A subscriber on `<path>/**` can use it to detect dropped or reordered updates.

Zenoh delivers a node's own puts back to its own matching subscribers, so a node must **discard samples whose `(source_id.zid, source_id.eid)` matches the publisher it declared for that object**. Don't compare `zid` alone. Many objects (or an authoritative object and a mirror of another one) can share a session, and comparing only `zid` would make them ignore each other's genuine updates.

Applying a received update must not trigger a re-publish of that update.

5.7 6. Built-in queryables

Every authoritative object implements **three callables**. Like any other method (see §7), each one is a queryable under the object's path, and two of them are also mounted at extra keys:

Callable	Mounted at	Returns
`sr_rehydrate`	`<path>/sr_rehydrate`	the object's full state
`sr_metadata`	`<path>/sr_metadata`, plus `<path>/sr_metadata/<Tag>` for each tag in the class's MRO	address, tags and owner
`sr_object_sc`	`<path>/sr_object_schema`, plus `<path>/sr_type_schema/<Tag>` for each tag in the class's MRO	the object's *schema*

Every key that a callable is mounted at runs the same code and returns the same reply. `<Tag>` is a YAML tag without its `!`. For example, a `Probe(SyncableObject)` at `myhost/probe` answers on `…/sr_metadata/Probe`, `…/sr_metadata/SyncableObject`, `…/sr_type_schema/Probe` and `…/sr_type_schema/SyncableObject`. The per-tag keys let a client filter by type, including parent types, with a single Zenoh wildcard query (see *§8*).

All three reply with an `application/yaml` payload, using the queryable's own key as the reply key, so a wildcard query always shows which object answered. They take no parameters, and passing any is an error. They're listed in the object's own `x-rpc-endpoints` alongside its other methods (see *§9*), so a client can check what a node actually implements rather than relying on this page.

5.7.1 `sr_rehydrate`

Returns the full current state of the object: a mapping of every synced field, tagged with the object's YAML tag. The mapping also carries `synq_topic` and `synq_base_topic`. These are the object's address, they aren't in the schema, and clients should ignore them along with any other keys they don't know.

```
!Probe
blob: !!binary |
  AAE=
gimbal: null
lit: a
maybe: null
name: x
pt: null
res:
- 1
- 2
speed: 0.0
st: !!set {}
synq_base_topic: myhost
synq_topic: probe
tags: []
```

A mirror calls this once when it attaches. It also calls it to recover from a *missing parent*.

5.7.2 `sr_metadata`

```yaml
authoritive_node: abdc77db26c912dc6a6ada9fc4b056a3   # Zenoh session ZID␣
↪(spelling is intentional)
classes:
- '!Probe'
- '!SyncableObject'
topic: myhost/probe
```

`classes` lists every YAML tag in the MRO, sorted.

5.7.3 `sr_object_schema` and `sr_type_schema/<Tag>`

Returns the object's schema, described in *§9*. The `sr_type_schema/<Tag>` keys are the same callable, so `**/sr_type_schema/Gimbal` gets one reply from **each object** whose class is or inherits from `Gimbal`, keyed by that object's path. If two objects of the same type return different schemas (for example because they run different versions of the code), that's a real disagreement, and the client sees both.

Types that only appear nested inside an object, such as a dataclass field's type, don't get keys of their own. Their definitions are in the `$defs` of every object schema that uses them.

5.8 7. RPC methods

Each `@remote_method` is a queryable at `<path>/<method_name>`.

5.8.1 Request

Arguments are passed **by name** as Zenoh selector parameters. Each value is a YAML document with the trailing `\n...` stripped:

```
myhost/probe/add?a=2;b=40
```

- **Parameters are separated by `;`, not `&`.** This is the Zenoh 1.x selector syntax. With `?a=2&b=40`, the server sees a single parameter `a` with the value `2&b=40`.
- **Values are percent-encoded for `%` and `;`.** Zenoh has no escaping of its own, so a raw `;` inside a value would split it. Clients must write `%` as `%25` and `;` as `%3B` (other characters may be percent-encoded too), and servers percent-decode each value before parsing it as YAML. For example, the string `a;b` is sent as `?v=a%3Bb`.
- `&`, `=`, `|` and `#` are all fine inside a value.
- A request whose parameters fail to parse gets an error reply, not silence.
- The reference client sends every parameter, defaults included. Omitted parameters take the method's default. Unknown parameters are an error.
- The Zenoh-level `ConsolidationMode` must be `NONE` for *generator methods*. For other methods it doesn't matter.

5.8.2 Response

The return value is a single reply, encoded as YAML with encoding `application/yaml`. A method that returns nothing replies `null`.

5.8.3 Errors

An exception is sent as a Zenoh **error reply** (`reply_err`). Its payload is a UTF-8 string of the form `RPC error '<message>'`, sent with encoding `zenoh/bytes`. Clients should treat the payload as a human-readable string and not parse it.

5.8.4 Generator methods

A generator method sends **several replies** to a single query:

1. one `application/yaml` reply for each yielded value, in order;
2. a final reply with encoding **`x-spirisynq/generator-done`**. Its YAML payload is the generator's return value (`null` for async generators, which can't return a value).

```
GET myhost/probe/count?n=3     (ConsolidationMode.NONE)
  → application/yaml               0
  → application/yaml               1
  → application/yaml               2
  → x-spirisynq/generator-done     done
```

If the generator raises an exception partway through, the stream ends with an error reply instead of the done reply. A client **must** request `ConsolidationMode.NONE`. Zenoh's default consolidation keeps only the last reply for each key, which would drop every yielded value. The schema marks generator methods with `x-generator: true` (see *§9*).

5.9 8. Discovery

To find	Query (`ConsolidationMode.NONE`)	One reply per
every object	`**/sr_metadata`	object
objects of a type (or subtype)	`**/sr_metadata/<Tag>`, e.g. `**/sr_metadata/Counter`	matching object
schemas of every object of a type	`**/sr_type_schema/<Tag>`	matching object
anything under a prefix	`<prefix>/**/…` in place of `**/…`	

Don't query `**/sr_metadata/` (with a trailing slash): it isn't a valid key expression. Don't use `**/sr_metadata/**` either, because it matches the bare key and every per-tag key, so you get each object once per tag. The same applies to `sr_type_schema`.

5.10 9. Object schema

`sr_object_schema` returns a **standard JSON Schema (draft 2020-12)** document describing one object. Every schema the reference implementation produces passes the 2020-12 meta-schema, and the test suite validates real `sr_rehydrate` payloads against it. SpiriSynq-specific information is carried in `x-` keywords, which standard validators ignore. The document is sent as YAML, but it contains only JSON values, so a consumer can treat it as JSON.

The schema does three jobs:

- it describes the **field values** that can arrive at `<path>/<field>` and inside `sr_rehydrate`;
- it says how each value is **encoded** on the wire (YAML tags and non-YAML encodings);
- it lists **every callable** the object serves, including the three built-ins, with their parameters and results, in `x-rpc-endpoints`.

Generic clients use it. `Session.from_topic_untyped` reads the `properties` names, the CLI's `synq topic rpc` reads `x-rpc-endpoints`, and `synq meta type_schema` prints `sr_type_schema` replies.

5.10.1 Document structure

```
$schema: https://json-schema.org/draft/2020-12/schema
x-spirisynq-schema: 2                    # format version, see below
title: <class name>
type: object
x-yaml-tag: '!<Tag>'                     # the tag the object is sent with
x-classes: ['!<Tag>', ...]               # every tag in the MRO, sorted (as in␣
↪sr_metadata)
description: <class docstring>           # omitted if the class has none
properties:
  <field>: <value schema>                # one per synced top-level field
required: [<every field>]
x-rpc-endpoints:
  <callable>: <endpoint>                 # see below
$defs:                                   # omitted when nothing is referenced
  <ClassName>: <object schema>
```

- **`properties`** lists every top-level field that can appear in field puts. It leaves out _-prefixed fields, `synq_*` settings and addressing (`synq_topic`, `synq_base_topic`), `ClassVars` (so `<field>_qos` never appears) and anything in `synq_skip_sync`.
- **`required`** lists every field, because `sr_rehydrate` always sends all of them. Whether a field can be `null` is part of its value schema, not of `required`.
- There's no `additionalProperties: false`, because `sr_rehydrate` also carries the addressing fields (see *§6*).
- **`description`** comes from the class's own docstring, cleaned with `inspect.cleandoc`. The signature-style docstring that `@dataclass` generates for an undocumented class is left out, and so are inherited docstrings. A field's `description` comes from `field(metadata={"help": "..."})`. Comments next to a field aren't visible at runtime, so put units and similar information in `help`.

- **`default`** on a property is the field's Python default, converted to JSON (see *Defaults*).

`x-spirisynq-schema` is `2` for the format described here. Version 1, sent by releases up to 0.2.0, had no version marker and used non-standard `type` names. A consumer that finds no `x-spirisynq-schema` should treat the document as version 1 and not validate with it.

5.10.2 Type mapping

Python annotation	Value schema
`int` / `float` / `str` / `bool`	`{type: integer}` / `number` / `string` / `boolean`
`X | None`	`X` with `null` added: `{type: [integer, 'null']}`, or `{anyOf: [<X>, {type: 'null'}]}` when `X` is a `$ref` or a union
`X | Y`	`{anyOf: [<X>, <Y>]}`
`Literal["a", "b"]`	`{type: string, enum: [a, b]}`. Mixed value types give a list of types. `Literal[...] | None` adds `null` to both `type` and `enum`
`list[X]`	`{type: array, items: <X>}`
`tuple[X, Y]`	`{type: array, prefixItems: [<X>, <Y>], items: false, minItems: 2}`
`tuple[X, ...]`	`{type: array, items: <X>}`
`set[X]`, `frozenset[X]`	`{type: array, uniqueItems: true, items: <X>, x-yaml-tag: '!!set'}`
`dict[K, V]`	`{type: object, additionalProperties: <V>}` (keys aren't described)
`bytes`	`{type: string, contentEncoding: base64, x-yaml-tag: '!!binary'}`
`EventedList` / `EventedSet` / `EventedDict`	`array` / `array` with `uniqueItems` / `object`, with `x-yaml-tag: '!EventedList'` etc.
`Link`	`{type: string, x-yaml-tag: '!Link'}`
an `Enum` class	`{enum: [<member values>], x-python-type: <name>}`
a `str`/`int`/`float` subclass with a `yaml_tag` (e.g. `RootFrame`)	the base type plus `x-yaml-tag`
a dataclass (including `SubSyncableDataclass`) or `TypedDict`	`{$ref: '#/$defs/<ClassName>'}`
`Any`, `object`, or an annotation that can't be resolved	`{}` (any value)
any other class	`{x-python-type: <name>}`, plus `x-yaml-tag` if it has one: an opaque value that accepts anything

`$defs` has one entry per referenced dataclass or `TypedDict`, keyed by class name. A dataclass entry is an object schema with `title`, `x-yaml-tag`, `properties` and `required` (every field). A `TypedDict` entry has no `x-yaml-tag`, because it's sent as a plain mapping, and its `required` follows the TypedDict's own required keys. Recursive types terminate, because each type is defined once.

5.10.3 Wire annotations

These keywords tell a receiver how a value looks on the wire. Standard validators ignore them.

Keyword	Meaning
`x-yaml`	The YAML tag the value is sent with. On an object schema it's the object's own tag (`!Probe`). On a value schema it's the tag a nested value carries (`!Gimbal`, `!!set`, `!!binary`, `!EventedList`).
`x-enco`	Field puts for this property **aren't YAML**. They use a codec with this Zenoh encoding (for example `zenoh/bytes` for `bytes`, see *§2*). It reflects the codecs registered on the serving node. Inside `sr_rehydrate` and RPC payloads the value is still YAML.
`x-pyth`	The Python class name of a value the schema can't describe structurally. Useful for display and code generation only.
`x-clas`	Root only: the object's tags across its MRO, the same list `sr_metadata` returns.

5.10.4 Defaults

A `default` is included when the Python default can be expressed as JSON:

- primitives, and `None` as `null`;
- enum members, as their value;
- `bytes`, as base64;
- sets, as sorted arrays;
- frozen dataclass instances, as objects.

A `default_factory` is called only if it's a builtin container or scalar constructor (`list`, `dict`, `set`, `EventedList` and similar). Other factories are never run, and their fields simply have no `default`.

5.10.5 `x-rpc-endpoints`

There's one entry for each callable the object serves, keyed by the name it's served under (the last key chunk). This includes `sr_rehydrate`, `sr_metadata` and `sr_object_schema`, so an object fully describes itself. Extra mount points (`sr_metadata/<Tag>`, `sr_type_schema/<Tag>`) aren't listed separately.

```
<callable>:
  description: <docstring, cleaned>       # omitted if none
  parameters:                             # always present: a JSON Schema for␣
↪the selector parameters
    type: object
    properties:
      <name>: <value schema>              # {} if unannotated; `default` if it␣
↪has one
    required: [<parameters without a default>]
    additionalProperties: false           # unknown parameters are an error
  returns: <value schema>                 # {type: 'null'} for -> None; {} if␣
↪unannotated
```

(continues on next page)

(continued from previous page)

```
  x-generator: true                          # generator methods only (see §7)
  yields: <value schema>                     # generator methods only: each␣
↪streamed value
```

- To validate a call, validate the selector parameters, decoded from YAML into a mapping, against `parameters`.
- For a generator, `returns` describes the payload of the final `x-spirisynq/generator-done` reply. It's taken from `Generator[Y, S, R]`. For `Iterator[Y]`, `AsyncGenerator[Y, S]` and other generators that can't return a value, it's `null`.
- `returns: {$ref: '#'}` means "an object of this schema" (`sr_rehydrate`). `returns: {$ref: 'https://json-schema.org/draft/2020-12/schema'}` means "a JSON Schema document" (`sr_object_schema`).
- `$ref`s in parameters and results point into the same top-level `$defs`.

5.10.6 Worked example

This class:

```python
@dataclass
class Gimbal(SubSyncableDataclass):
    """A gimbal."""
    pitch: float = 0.0

@dataclass
class Probe(SyncableObject):
    """A probe."""
    speed: float = field(default=0.0, metadata={"help": "metres per second"})
    mode: Literal["idle", "run"] = "idle"
    count: int | None = None
    blob: bytes | None = None
    gimbal: Gimbal | None = None

    @remote_method()
    def set_speed(self, value: float, ramp: bool = False) -> None:
        """Set the speed."""

    @remote_method()
    def countdown(self, n: int) -> Generator[int, None, str]:
        """Count down from n."""
```

serves this from `myhost/probe/sr_object_schema`, and from `myhost/probe/sr_type_schema/Probe` and `…/SyncableObject`. The reply is shown here sorted by key and lightly reflowed:

```
$defs:
  Gimbal:
```

(continues on next page)

(continued from previous page)

```
    title: Gimbal
    type: object
    x-yaml-tag: '!Gimbal'
    description: A gimbal.
    properties:
      pitch: {default: 0.0, type: number}
    required: [pitch]
  SyncableObjectMetadata:                    # result of sr_metadata
    title: SyncableObjectMetadata
    type: object
    description: "Reply of sr_metadata: an object's address, YAML tags, and␣
↪owner."
    properties:
      authoritive_node: {type: string}
      classes: {items: {type: string}, type: array}
      topic: {type: string}
    required: [authoritive_node, classes, topic]
$schema: https://json-schema.org/draft/2020-12/schema
description: A probe.
properties:
  blob:
    type: [string, 'null']
    contentEncoding: base64
    x-yaml-tag: '!!binary'
    x-encoding: zenoh/bytes                  # field puts are raw bytes, not␣
↪YAML
    default: null
  count: {default: null, type: [integer, 'null']}
  gimbal:
    anyOf: [{$ref: '#/$defs/Gimbal'}, {type: 'null'}]
    default: null
  mode: {default: idle, enum: [idle, run], type: string}
  speed: {default: 0.0, description: metres per second, type: number}
required: [blob, count, gimbal, mode, speed]
title: Probe
type: object
x-classes: ['!Probe', '!SyncableObject']
x-rpc-endpoints:
  countdown:
    description: Count down from n.
    parameters:
      type: object
      properties: {n: {type: integer}}
      required: [n]
      additionalProperties: false
    x-generator: true
```

(continues on next page)

(continued from previous page)

```
    yields: {type: integer}
    returns: {type: string}
  set_speed:
    description: Set the speed.
    parameters:
      type: object
      properties:
        ramp: {default: false, type: boolean}
        value: {type: number}
      required: [value]
      additionalProperties: false
    returns: {type: 'null'}
  sr_metadata:
    description: Returns topic path, YAML type tags, and authoritative node ID.
    parameters: {type: object, properties: {}, required: [],␣
↪additionalProperties: false}
    returns: {$ref: '#/$defs/SyncableObjectMetadata'}
  sr_object_schema:
    description: Returns the JSON Schema for this object's syncable fields and␣
↪RPC endpoints.
    parameters: {type: object, properties: {}, required: [],␣
↪additionalProperties: false}
    returns: {$ref: 'https://json-schema.org/draft/2020-12/schema'}
  sr_rehydrate:
    description: Returns the full current state of this object.
    parameters: {type: object, properties: {}, required: [],␣
↪additionalProperties: false}
    returns: {$ref: '#'}
x-spirisynq-schema: 2
x-yaml-tag: '!Probe'
```

5.11 Implementer's checklist

Authoritative node, for one object at `P`:

- [ ] Implement `sr_rehydrate`, `sr_metadata` and `sr_object_schema`, and mount them at `P/sr_rehydrate`, `P/sr_metadata`, `P/sr_object_schema`, plus `P/sr_metadata/<Tag>` and `P/sr_type_schema/<Tag>` for each tag in the MRO
- [ ] Declare `P/<method>` for each RPC method. Parse `;`-separated selector parameters, decode each one as YAML, and reply `application/yaml`, or send a `reply_err` string
- [ ] For generator methods, send one reply per value, then a final `x-spirisynq/generator-done` reply
- [ ] Describe every callable, including the three built-ins, in `x-rpc-endpoints`
- [ ] Declare a publisher on `P/**`. Send each field change as a put at `P/<field path>` with `Source-`

`Info(publisher id, per-publisher counter)`

- [ ] On shutdown, send a reliable delete on `P/**` from the same publisher, with the next `SourceInfo` in its sequence
- [ ] Subscribe to `P/**`, apply puts from other nodes, and discard samples whose `(zid, eid)` is your publisher's

Mirror, for an object at `P`:

- [ ] Subscribe to `P/**` **before** you rehydrate, so no update is lost in between
- [ ] Query `P/sr_rehydrate`, then apply field puts as they arrive. Choose the decoder by the sample's encoding, using `x-encoding` from the schema to know which fields aren't YAML
- [ ] To write, put to `P/<field>` with your own `SourceInfo`, and discard your own echoes
- [ ] Mark the object deleted when a delete on `P/**` or `P` arrives
- [ ] Discover objects with `**/sr_metadata[/<Tag>]` and `ConsolidationMode.NONE`

CHAPTER

SIX

API REFERENCE

Auto-generated from source. For conceptual documentation see the main docs.

6.1 Session

class SpiriSynq.session.**Session**(*config=<factory>*, *base_topic=<factory>*, *type_registry=<factory>*, *objects=<factory>*, *_sequence_number_for_source=<factory>*, *_codecs=<factory>*, *_generic_classes=<factory>*)

Bases: `object`

A connection to the Zenoh network.

Most applications use the module-level default session created at import time and never construct a `Session` directly. You only need to create one explicitly if you want to connect with custom Zenoh config, or to connect to multiple independent Zenoh networks from the same process.

The session opens a Zenoh peer connection on construction and closes it on garbage collection. For deterministic cleanup, call *`close()`* explicitly.

Example — custom config:

```
import zenoh
from SpiriSynq.session import Session

config = zenoh.Config.from_file("zenoh.json5")
session = Session(config=config)
counter = Counter("myapp/counter", synq_session=session,␣
↪synq_authoritive=True)
```

Example — multiple sessions:

```
session_a = Session()
session_b = Session()

with session_b.as_default():
    # Objects created here use session_b
    mirror = Counter.from_topic("myhost/myapp/counter")
```

Parameters

- **config** (*Config*)
- **base_topic** (*str*)
- **type_registry** (*SessionSerializer*)
- **objects** (*WeakValueDictionary*)
- **_sequence_number_for_source** (*dict[tuple, int]*)
- **_codecs** (*list*)
- **_generic_classes** (*dict*)

config: Config

Zenoh configuration. Defaults to peer-mode auto-discovery with no router.

base_topic: str

Topic prefix prepended to all authoritative objects on this session. Defaults to, in order: the `SPIRI_SYNQ_BASE_TOPIC` environment variable, the contents of `/etc/spirisynq_base_topic` (e.g. the host's `/etc/hostname` bind-mounted into a container), then the hostname.

type_registry: SessionSerializer

YAML serialiser/deserialiser used for all payloads on this session. Types are registered here by *register_type_recursive()*.

zenoh_session: *Session*

The underlying Zenoh session. Available after construction.

objects: WeakValueDictionary

Weak map of `absolute_path → SyncableObject` for all live objects on this session.

as_default()

Context manager that makes this session the default for the current thread/task.

Any `SyncableObject` constructed inside the block that does not pass `synq_session=` explicitly will use this session instead of the module-level default.

Can also be used as a plain context manager around a block of code:

```
session2 = Session()
with session2.as_default():
    mirror = Counter.from_topic("myhost/myapp/counter")
    # mirror was created on session2
```

Parameters

self (Session)

is_class_registered(*cls*)

Return True if *cls* has been registered with this session's type registry.

resolve_topic(*topic*)

Expand a leading `.` chunk to *base_topic* and validate the result.

`./camera` resolves to `<base_topic>/camera` and `.` alone to the base topic itself, so `./camera` and `myhost/camera` name the same key. Topics without a leading `.` are returned unchanged. `.` anywhere else, and `..` anywhere, raise `ValueError` — `.` is a valid literal Zenoh chunk, so an unexpanded marker must never reach the network.

Parameters

topic (*str*)

Return type

str

from_topic(*topic*)

Fetch and return the current state of an object at *topic* via `sr_rehydrate`.

The type must already be registered with this session (call *register_type_recursive()* first, or use `SyncableObject.from_topic` which handles registration automatically).

Returns the deserialised object. The concrete type depends on the YAML tag in the reply, so the return type is `Any`. *topic* may be relative (`./camera`); see *resolve_topic()*.

from_topic_untyped(*topic*)

Mirror an object at *topic* without its dataclass being importable here.

Discovers the object's field names (`sr_object_schema`) and wire type tags (`sr_metadata`), synthesizes a matching `SyncableObject` subclass with every field typed `object`, registers it for those tags, and returns a mirror – same as *from_topic()*, but for types this process never defined or imported (e.g. a peer written in another language, or a dynamically-defined class on the authoritative side).

Nested custom dataclass fields still decode to whatever type tag they carry on the wire; if that type also isn't registered here, decoding those nested values will fail. Only the object's own top-level fields are guaranteed to work generically.

Parameters

topic (*str*)

list_topics(*type_filter=''*, *prefix=''*)

Yield metadata dicts for all objects currently discoverable on the network.

Queries `**/sr_metadata/<type_filter>` and yields one dict per reply. Each dict contains at minimum `topic`, `classes`, and `authoritive_node`.

Parameters

- **type_filter** (*str*) – If given, only topics whose class list includes this type name (without the `!` YAML tag prefix) are returned.
- **prefix** (*str*) – Restrict the search to topics under this key prefix. May be relative (`.` for this session's own objects).

Example:

```
for meta in session.list_topics(type_filter="Counter"):
    print(meta["topic"])
```

Works identically to `synq topic list` on the CLI.

register_type_recursive(*cls*)

Register *cls* and every type reachable from its field annotations.

Walks all type hints transitively. For each type:

- Already representable (via MRO against the YAML representer) → skip.
- Otherwise → assign a yaml_tag if missing, call register_class, then verify it took. Raises TypeError if a type cannot be registered so the problem surfaces at sync() time rather than at first serialisation.

Parameters

cls (*type*)

Return type

None

register_codec(*codec*)

Register a custom `Codec` on this session.

The encoder is selected by matching `codec.python_type` against the Python type of the value being published (MRO walk, most-specific first). The decoder is selected by matching `codec.zenoh_schema` against the encoding reported by an incoming zenoh sample.

Example:

```
import numpy as np
import zenoh
from SpiriSynq.session import Codec, current_session

session = current_session.get()
session.register_codec(Codec(
    python_type=np.ndarray,
    zenoh_schema=zenoh.Encoding("image/jpeg"),
    encoder=lambda arr: (encode_jpeg(arr), zenoh.Encoding("image/jpeg
↪")),
    decoder=lambda sample: decode_jpeg(sample.payload.to_bytes()),
))
```

Parameters

codec (*Codec*)

Return type

None

source_info(*source_id=None*)

Build a `zenoh.SourceInfo` for the next sample from *source_id*.

As with zenoh's own `source_sn`, the sequence number is monotonic per source entity (zid + eid), not per key, so a subscriber that sees all of a publisher's samples can detect gaps and reordering. Each `SyncableObject` publishes under its own publisher's id, so objects on the same session don't share a counter.

close()

6.2 Schema

JSON Schema (2020-12) generation for synced dataclasses.

The format is part of the wire protocol (`sr_object_schema` and `sr_type_schema`); see docs/protocol.md before changing it, and bump `SCHEMA_VERSION` for incompatible changes.

SpiriSynq.schema.**SCHEMA_VERSION = 2**

Value of `x-spirisynq-schema` on every root schema.

class SpiriSynq.schema.**JsonSchema**

Return annotation for methods that return a JSON Schema document.

alias of `dict`

SpiriSynq.schema.**get_schema**(*cls*, *codecs=None*)

Generate the JSON Schema for a dataclass, TypedDict or SyncableObject.

codecs is the session's codec list, used to mark fields whose puts are not YAML (`x-encoding`); defaults to the built-in codecs.

Parameters

cls (*type*)

Return type

dict

SpiriSynq.schema.**yaml_tag**(*t*)

The YAML tag a dataclass is sent with, as `register_type_recursive` assigns it.

Parameters

t (*type*)

Return type

str

6.3 Remote Callables

exception SpiriSynq.remote_callables.**RpcException**(*message*, *topic=None*)

Bases: `Exception`

Parameters

- **message** (*str*)
- **topic** (*str | None*)

SpiriSynq.remote_callables.**encode_parameters**(*params*)

Build selector parameters from already-serialised values.

Zenoh splits parameters on `;` and has no escaping of its own, so `%` and `;` inside a value are percent-encoded. *decode_parameters()* undoes it.

Parameters

params (*Dict[str, str]*)

Return type

Parameters

SpiriSynq.remote_callables.**decode_parameters**(*query*)

Return a query's parameters as raw (still YAML-encoded) strings.

Parameters

query (*Query*)

Return type

Dict[str, str]

SpiriSynq.remote_callables.**rpc_call**(*topic*, *session*, *kwargs=None*, *retries=3*)

Parameters

- **topic** (*str*)
- **session** (Session)
- **kwargs** (*Dict[str, Any] | None*)
- **retries** (*int*)

class SpiriSynq.remote_callables.**BoundRemoteMethod**(*remote_method*, *instance*, ** (Keyword-only parameters separator (PEP 3102))*, *_timeout=None*, *_retries=3*)

Bases: object

A RemoteMethod bound to a specific instance, exposing both sync and async interfaces.

Parameters

- **remote_method** (RemoteMethod)
- **instance** (*Any*)
- **_timeout** (*float | None*)
- **_retries** (*int*)

timeout(*seconds*)

Parameters

seconds (*float*)

Return type

BoundRemoteMethod

retries(*n*)

Parameters

n (*int*)

Return type

BoundRemoteMethod

sync(**args*, ***kwargs*)

as_async(**args*, ***kwargs*)

Return a coroutine (regular methods) or async generator (generator methods).

setup_zenoh_callback(*parent*, *path=None*, *name=None*)

Parameters

- **parent** (SyncableObject)
- **path** (*str | None*)
- **name** (*str | None*)

class SpiriSynq.remote_callables.**RemoteMethod**(*wrapped*)

Bases: object

Parameters

wrapped (*Callable[[...], T]*)

client(*, *raw=False*)

Parameters

raw (*bool*)

Return type

Callable

server()

Return type

Callable

setup_zenoh_callback(*parent*, *path=None*, *name=None*)

Parameters

- **parent** (SyncableObject)
- **path** (*str | None*)

- **name** (*str | None*)

SpiriSynq.remote_callables.**undeclare**(*key*, *queryable*)

SpiriSynq.remote_callables.**remote_method**()

Return type
Callable[[*Callable*[[…], *T*]], RemoteMethod]

6.4 Syncable Objects

SpiriSynq.syncable_objects.**SKIP = SKIP**

Returned from a `<field>_receive` hook to leave the field unchanged.

class SpiriSynq.syncable_objects.**PutArgs**

Bases: TypedDict

One put yielded by a `<field>_publish` hook: keyword arguments for `zenoh.Session.put`, minus the key and `source_info`, which SpiriSynq sets. QoS keys left out come from the field's `<field>_qos`. Annotate a hook as `-> Iterator[PutArgs | bytes | str]` to have these keys checked.

payload: Required[bytes | bytearray | str | ZBytes]

encoding: Encoding | str

congestion_control: CongestionControl

priority: Priority

express: bool

attachment: bytes | bytearray | str | ZBytes

timestamp: Timestamp

allowed_destination: Locality

class SpiriSynq.syncable_objects.**SyncableObjectMetadata**

Bases: TypedDict

Reply of sr_metadata: an object's address, YAML tags, and owner.

topic: str

classes: list[str]

authoritive_node: str

class SpiriSynq.syncable_objects.**SubSyncableDataclass**

Bases: object

Lightweight base for nested sub-objects within a SyncableObject.

Provides psygnal events, synq_skip_sync, and synq_skip_rehydrate filtering without any zenoh publish/subscribe machinery. Use this instead of @dataclass(frozen=True) when you need mutable nested objects whose field changes fire events (and therefore propagate through the parent's sync).

events: ClassVar[SignalGroupDescriptor]

synq_skip_rehydrate: ClassVar[set] = {}

synq_skip_sync: ClassVar[set] = {}

classmethod all_skip_rehydrate()

Return type
set

classmethod valid_sync_paths()

Return type
frozenset

classmethod to_yaml(*representer*, *data*)

class SpiriSynq.syncable_objects.**WeakMethodProxy**(*method*, *callback=None*)

Bases: object

class SpiriSynq.syncable_objects.**SyncableObject**(*synq_topic: str*, *synq_base_topic: str | None = None*, *synq_session: SpiriSynq.session.Session | None = <factory>*, *synq_authoritive: bool = False*, *synq_lazy_publish: bool = False*, *synq_publish: bool = True*, *synq_receive: bool = True*, *synq_auto_start: bool = True*, *synq_check_receive_types: bool = True*, *synq_auto_rehydrate_on_missing_parent_timeout: float = 5.0*, *synq_mtime: float = -1*, *synq_is_deleted: bool = False*, *_synq_callbacks: dict = <factory>*, *_synq_last_rehydrate_time: float = 0.0*)

Bases: object

Parameters

- **synq_topic** (*str*)
- **synq_base_topic** (*str | None*)
- **synq_session** (*Session | None*)

- **synq_authoritive** (*bool*)
- **synq_lazy_publish** (*bool*)
- **synq_publish** (*bool*)
- **synq_receive** (*bool*)
- **synq_auto_start** (*bool*)
- **synq_check_receive_types** (*bool*)
- **synq_auto_rehydrate_on_missing_parent_timeout** (*float*)
- **synq_mtime** (*float*)
- **synq_is_deleted** (*bool*)
- **_synq_callbacks** (*dict*)
- **_synq_last_rehydrate_time** (*float*)

events: ClassVar[SignalGroupDescriptor]

synq_topic: str

synq_base_topic: str | None = None

synq_session: *Session* | None

synq_authoritive: bool = False

synq_lazy_publish: bool = False

Only publish changes if there are active subscribers on the zenoh network. Useful for reducing unnecessary network traffic when no one is listening.

synq_publish: bool = True

Whether this object should publish its changes to zenoh. Set to False to make this object receive-only.

synq_receive: bool = True

Whether this object should apply incoming changes from zenoh. Set to False to make this object publish-only.

synq_auto_start: bool = True

Automatically call sync() in __post_init__. Set to False if you need to configure the object before starting synchronization.

synq_check_receive_types: bool = True

Validate that received values match the expected type annotation before applying them. Logs a warning and skips the update on type mismatch.

synq_auto_rehydrate_on_missing_parent_timeout: float = 5.0

Seconds between automatic sr_rehydrate calls triggered by missing-parent updates. Set to -1 to disable.

synq_mtime: float = -1

time.monotonic() timestamp of the last successfully applied remote update. Only updated on receive — never published or synced (monotonic clocks are process-local and meaningless across nodes). -1 if no remote update has been received yet.

synq_signal_unknown_path

Emitted when a received update targets a path not in valid_sync_paths(). Args: (relative_path, zenoh.Sample) — object is not decoded since the path is unknown.

synq_signal_type_mismatch

Emitted when a received value's type doesn't match the field annotation. Args: (relative_path, decoded_obj)

synq_signal_missing_parent

Emitted when a nested path is valid but an intermediate object is None (e.g. path is 'bar/value' but self.bar is None). Args: (relative_path, decoded_obj)

synq_signal_tombstone

Emitted when a tombstone (delete) is received for this object's base path, indicating the authoritative node has deleted it.

synq_skip_rehydrate = {}

synq_is_deleted: bool = False

True after a tombstone has been received or sent (authoritative close).

synq_skip_sync = {'_synq_callbacks', '_synq_last_rehydrate_time', 'synq_authoritive', 'synq_auto_rehydrate_on_missing_parent_timeout', 'synq_auto_start', 'synq_check_receive_types', 'synq_is_deleted', 'synq_lazy_publish', 'synq_mtime', 'synq_publish', 'synq_receive', 'synq_session'}

sync()

Set up hookes for synchronization, including any relevent RPC hooks

synq_qos_for(*path*)

Resolve the QoS for a sync path, or None for zenoh's defaults.

Checks `<field>_qos` on the object that owns the field (instance attribute, then class attribute), then the nearest ancestor field (`bar/value` falls back to `bar_qos`). Only SynqQoS instances count.

Parameters

path (*str*)

Return type

SynqQoS | None

classmethod from_topic(*topic*, *session=None*)

property synq_absolute_path: str

classmethod synq_type_tags()

Return type
list

sr_metadata()

Returns topic path, YAML type tags, and authoritative node ID.

Return type
SyncableObjectMetadata

sr_rehydrate()

Returns the full current state of this object.

Return type
Self

sr_rehydrate_client()

Returns the full current state of this object.

Return type
Self

sr_object_schema()

Returns the JSON Schema for this object's syncable fields and RPC endpoints.

Return type
JsonSchema

classmethod all_skip_rehydrate()

Returns synq_skip_rehydrate on this and all parent classes merged

Return type
set

classmethod valid_sync_paths()

Returns all valid Zenoh publish paths for this class. Merges synq_skip_sync across the MRO, then recursively walks dataclass field type annotations. Cached on the class itself.

Return type
frozenset[str]

classmethod valid_sync_type(*path*, *obj*)

Returns True if obj matches the expected type(s) at the given sync path.

Parameters

- **path** (*str*)
- **obj** (*object*)

Return type
bool

classmethod to_yaml(*representer*, *data*)

sync_dumps()

Canonical yaml representation of this object

Return type
str

close()

Undeclare all zenoh resources and disconnect event handlers.

SpiriSynq.syncable_objects.**make_generic_syncable_class**(*name*, *field_names*)

Build a SyncableObject subclass with *field_names*, each typed `object`.

Used by `Session.from_topic_untyped` to mirror objects whose real dataclass isn't importable in this process (e.g. a peer written in another language, or a class this process never imported). Skips names that collide with SyncableObject's own fields, since those are always supplied positionally/by keyword by the sync machinery itself.

Parameters

- **name** (*str*)
- **field_names** (*list*)

Return type
type

6.5 Link

class SpiriSynq.link.**Link**(*topic*)

Bases: `object`

A reference to another SpiriSynq object by its absolute topic.

Sent as a tagged scalar: `!Link myhost/camera`. A link holds only the topic – it doesn't keep the target alive or check that it exists until *resolve()* fetches it.

The topic must be absolute: a relative `./camera` would name a different object on every node that received it. Use *to()* to link to a live object.

Parameters
topic (*str*)

yaml_tag = '!Link'

topic: str

classmethod to(*obj*)

Link to *obj* at its absolute path.

Parameters
obj (SyncableObject)

Return type
Link

resolve(*cls=None*, *session=None*)

Fetch a mirror of the linked object.

With *cls*, this is `cls.from_topic(self.topic)`: it registers *cls* and checks the result is one. Without it, the target's type must already be registered on *session* (the current session by default). For a type this process doesn't have, use `session.from_topic_untyped(link.topic)`.

Parameters

- **cls** (*type[SyncableObject] | None*)
- **session** (*Session | None*)

classmethod to_yaml(*representer*, *data*)

classmethod from_yaml(*constructor*, *node*)

6.6 Example Types

6.6.1 Position

Positioned objects, as a tree of frames rooted in a *RootFrame*.

Frame conventions:

- **ECEF** (the usual root): metres; x through the equator at the prime meridian, y through the equator at 90°E, z through the north pole.
- **Body frames are FRD**: x forward, y right, z down – MAVLink's and aerospace's convention, so autopilot attitudes go in without conversion. A camera mounted 10 cm ahead of a drone's centre is `Offset(x=0.1)`.
- **Local level frames are NED** (x north, y east, z down); *Orientation.ned_to_ecef()* places one on the globe.
- An *Orientation* rotates vectors from the child's frame into its parent's, so `child.offset.rotate_by(parent.orientation)` is the child's offset expressed in the parent's parent.

class SpiriSynq.example_types.position.**RootFrame**(*value*)

Bases: `str`

A string identifying a known root coordinate frame.

Parameters

value (*str*)

Return type

RootFrame

yaml_tag = '!RootFrame'

classmethod is_known(*name*)

Parameters

name (*str*)

Return type
bool

classmethod to_yaml(*representer*, *data*)

classmethod from_yaml(*constructor*, *node*)

class SpiriSynq.example_types.position.**Offset**(*x=0*, *y=0*, *z=0*)

Bases: object

A displacement in metres along the axes of whatever frame holds it.

Parameters

- **x** (*float*)
- **y** (*float*)
- **z** (*float*)

x: **float** = **0**

y: **float** = **0**

z: **float** = **0**

classmethod from_geodetic(*lat*, *lon*, *height*)

ECEF offset of a WGS84 point.

lat/*lon* in degrees; *height* in metres above the ellipsoid. MAVLink altitudes are mostly MSL: add the geoid separation first (e.g. `GPS_RAW_INT.alt_ellipsoid - alt`).

Parameters

- **lat** (*float*)
- **lon** (*float*)
- **height** (*float*)

Return type
Offset

to_geodetic()

Inverse of *from_geodetic()* for an ECEF offset: (lat°, lon°, height above ellipsoid in m).

Return type
tuple[float, float, float]

rotate_by(*orientation*)

Return a new Offset rotated by the given orientation (quaternion).

Parameters
orientation (Orientation)

Return type
Offset

class SpiriSynq.example_types.position.**Orientation**(*w: float = 1*, *x: float = 0*, *y: float = 0*, *z: float = 0*)

Bases: object

Parameters

- **w** (*float*)
- **x** (*float*)
- **y** (*float*)
- **z** (*float*)

w: **float** = **1**

x: **float** = **0**

y: **float** = **0**

z: **float** = **0**

normalize()

Return type
Orientation

to_euler(*degrees=False*)

Returns (roll, pitch, yaw) in radians (or degrees if degrees=True). Uses ZYX intrinsic convention (yaw -> pitch -> roll).

Parameters
degrees (*bool*)

Return type
tuple[float, float, float]

property roll: **float**

property pitch: **float**

property yaw: **float**

classmethod from_euler(*roll*, *pitch*, *yaw*)

Construct from roll, pitch, yaw in radians. ZYX intrinsic convention.

Parameters

- **roll** (*float*)
- **pitch** (*float*)
- **yaw** (*float*)

Return type
Orientation

classmethod ned_to_ecef(*lat*, *lon*)

Orientation of the local NED frame at *lat*/*lon* (degrees) within ECEF.

Use it as the orientation of a NED frame's *Pose*, such as an autopilot's EKF origin, so children can mount in NED.

Parameters

- **lat** (*float*)
- **lon** (*float*)

Return type
Orientation

class SpiriSynq.example_types.position.**Accuracy**(*horizontal=None*, *vertical=None*)

Bases: object

Positional accuracy expressed as error bounds at 90% confidence.

There should be a 90% chance the object is inside of this cylinder.

Stored as CEP90/LEP90.

We break from ATAK/CoT here. On CoT ce/le uses 1-sigma (one standard deviation), which corresponds to ~39% confidence for horizontal (2D Rayleigh) and ~68% for vertical (1D Gaussian) — use at_confidence() to convert.

The intent behind this difference is that it's easy to convert, and provides developers with an easy and intuitive understanding of their confidence bounds. It roughly tracks to an intuitive "The object is here".

horizontal: radius in metres of the circle within which the true
horizontal position lies with 90% probability (CEP90).

vertical: half-height in metres of the vertical band within which
the true altitude lies with 90% probability (LEP90).

Parameters

- **horizontal** (*float | None*)
- **vertical** (*float | None*)

horizontal: float | None = None

vertical: float | None = None

at_confidence(*confidence*)

Return (horizontal, vertical) bounds rescaled to a different confidence level.

Stored values are CEP90/LEP90. Horizontal uses a 2D Rayleigh distribution; vertical uses a 1D symmetric Gaussian.

Parameters
confidence (*float*)

Return type
tuple[float | None, float | None]

to_cot()

Return (ce, le) in CoT 1-sigma convention.

Return type
tuple[float | None, float | None]

classmethod from_cot(*ce*, *le*)

Construct from CoT 1-sigma (ce, le) values.

Parameters

- **ce** (*float | None*)
- **le** (*float | None*)

Return type
Accuracy

class SpiriSynq.example_types.position.**Pose**(*offset=<factory>*, *orientation=<factory>*, *accuracy=None*)

Bases: object

Where something is and which way it faces, within some frame.

`orientation=None` makes it a point: something with a place but no facing, such as a region of interest. The default, the identity, means aligned with the parent frame.

One value, published as one put: a subscriber never pairs the offset from one update with the orientation from another.

Parameters

- **offset** (Offset)
- **orientation** (Orientation | *None*)
- **accuracy** (Accuracy | *None*)

offset: *Offset*

orientation: *Orientation* | None

accuracy: *Accuracy* | None = None

compose(*child*)

Place *child*, given in this pose's local frame, into the frame this pose is in.

The standard rigid-body chain, T_child = T_parent ∘ T_mount:

```
offset = parent.offset + child.offset.rotate_by(parent.orientation)
orientation = parent.orientation * child.orientation
```

Accuracies combine in quadrature; whichever is known propagates.

A point has no axes to measure from: under one, a child at a zero offset is a point at the same place, and any other child can't be placed, so this returns None. A point child stays a point.

Parameters
child (Pose)

Return type
Pose | None

classmethod from_plain(*data*)

Build from the tag-stripped mapping `load_untyped` gives for a Pose.

Parameters
data (*dict | None*)

Return type
Pose | None

class SpiriSynq.example_types.position.**Position**(*synq_topic*, *synq_base_topic=None*, *synq_session=<factory>*, *synq_authoritive=False*, *synq_lazy_publish=False*, *synq_publish=True*, *synq_receive=True*, *synq_auto_start=True*, *synq_check_receive_types=True*, *synq_auto_rehydrate_on_missing_parent_timeout=5.0*, *synq_mtime=-1*, *synq_is_deleted=False*, *_synq_callbacks=<factory>*, *_synq_last_rehydrate_time=0.0*, *relative_to='unknown'*, *mount=None*, *pose=None*, *_parent_subscriber=None*, *_parent_pose=None*, *_parent_live=False*, *_parent_generation=0*)

Bases: *SyncableObject*

Something with a place in the world: a node in a tree of frames.

`relative_to` names the parent: a *RootFrame*, or the topic of another Position: a *Link*, or a topic string (absolute, or relative like `./origin`). Prefer a Link: a relative topic means something different on every node that receives it. `mount` is this object's pose in its parent's frame – the value you set. `pose` is the resulting pose in the root frame, which the authoritative node computes and publishes; mirrors just receive it.

`pose` is None until it can be computed: without a mount, or while the parent's own pose is unknown. An unknown parent is never treated as the identity, which would put the object near the root frame's origin.

Parameters

- **synq_topic** (*str*)
- **synq_base_topic** (*str | None*)

- **synq_session** (Session | None)
- **synq_authoritive** (bool)
- **synq_lazy_publish** (bool)
- **synq_publish** (bool)
- **synq_receive** (bool)
- **synq_auto_start** (bool)
- **synq_check_receive_types** (bool)
- **synq_auto_rehydrate_on_missing_parent_timeout** (float)
- **synq_mtime** (float)
- **synq_is_deleted** (bool)
- **_synq_callbacks** (dict)
- **_synq_last_rehydrate_time** (float)
- **relative_to** (RootFrame | Link | str)
- **mount** (Pose | None)
- **pose** (Pose | None)
- **_parent_subscriber** (Subscriber | None)
- **_parent_pose** (Pose | None)
- **_parent_live** (bool)
- **_parent_generation** (int)

relative_to: *RootFrame* | *Link* | str = 'unknown'

mount: *Pose* | None = None

pose: *Pose* | None = None

close()

Undeclare all zenoh resources and disconnect event handlers.

SpiriSynq.example_types.position.**fuse_positions**(**positions*)

Fuse N independent position observations into a single best estimate.

Uses inverse-variance weighting when horizontal/vertical accuracies are known, falling back to a simple mean otherwise. More observations with known accuracy produce a tighter result. Positions whose pose is unknown are skipped.

Returns (offset, accuracy) where either component may have None fields if no accuracy information was available for that axis.

Parameters

positions (Position)

Return type
tuple[Offset, Accuracy]

CHAPTER

SEVEN

REVISION HISTORY

This table is generated from `revisions.yaml` at the root of this repository. Do not edit it here – add an entry there and it appears below, on the cover page, and in the PDF footer.

Revision	Date	Summary	Author
0.1 (draft)	2026-01-01	Initial draft.	Spiri Robotics

7.1 Issuing a revision

The document's revision is separate from the software's version: revise the document when what it says changes.

1. Add an entry to the bottom of `revisions.yaml`, without `draft: true`.
2. Merge it. The next software release (`vX.Y.Z` tag) attaches the PDF of this revision to its release. That release asset is the controlled copy: link people to it rather than to a PDF built from a branch, which can change under them.

PYTHON MODULE INDEX

S

SpiriSynq.example_types.position, 62
SpiriSynq.link, 61
SpiriSynq.remote_callables, 53
SpiriSynq.schema, 53
SpiriSynq.session, 49
SpiriSynq.syncable_objects, 56

INDEX

A

Accuracy (*class in SpiriSynq.example_types.position*), 65
accuracy (*SpiriSynq.example_types.position.Pose attribute*), 66
all_skip_rehydrate() (*SpiriSynq.syncable_objects.SubSyncableDataclass class method*), 57
all_skip_rehydrate() (*SpiriSynq.syncable_objects.SyncableObject class method*), 60
allowed_destination (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
as_async() (*SpiriSynq.remote_callables.BoundRemoteMethod method*), 55
as_default() (*SpiriSynq.session.Session method*), 50
at_confidence() (*SpiriSynq.example_types.position.Accuracy method*), 65
attachment (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
authoritive_node (*SpiriSynq.syncable_objects.SyncableObjectMetadata attribute*), 56

B

base_topic (*SpiriSynq.session.Session attribute*), 50
BoundRemoteMethod (*class in SpiriSynq.remote_callables*), 54

C

classes (*SpiriSynq.syncable_objects.SyncableObjectMetadata attribute*), 56
client() (*SpiriSynq.remote_callables.RemoteMethod method*), 55
close() (*SpiriSynq.example_types.position.Position method*), 68
close() (*SpiriSynq.session.Session method*), 53
close() (*SpiriSynq.syncable_objects.SyncableObject method*), 61
compose() (*SpiriSynq.example_types.position.Pose method*), 66
config (*SpiriSynq.session.Session attribute*), 50
congestion_control (*SpiriSynq.syncable_objects.PutArgs attribute*), 56

D

decode_parameters() (*in module SpiriSynq.remote_callables*), 54

E

encode_parameters() (*in module SpiriSynq.remote_callables*), 54
encoding (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
events (*SpiriSynq.syncable_objects.SubSyncableDataclass attribute*), 57
events (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
express (*SpiriSynq.syncable_objects.PutArgs attribute*), 56

F

from_cot() (*SpiriSynq.example_types.position.Accuracy class method*), 66
from_euler() (*SpiriSynq.example_types.position.Orientation class method*), 64
from_geodetic() (*SpiriSynq.example_types.position.Offset class method*), 63
from_plain() (*SpiriSynq.example_types.position.Pose class method*), 67

from_topic() (*SpiriSynq.session.Session method*), 51
from_topic() (*SpiriSynq.syncable_objects.SyncableObject class method*), 59
from_topic_untyped() (*SpiriSynq.session.Session method*), 51
from_yaml() (*SpiriSynq.example_types.position.RootFrame class method*), 63
from_yaml() (*SpiriSynq.link.Link class method*), 62
fuse_positions() (*in module SpiriSynq.example_types.position*), 68

G

get_schema() (*in module SpiriSynq.schema*), 53

H

horizontal (*SpiriSynq.example_types.position.Accuracy attribute*), 65

I

is_class_registered() (*SpiriSynq.session.Session method*), 50
is_known() (*SpiriSynq.example_types.position.RootFrame class method*), 62

J

JsonSchema (*class in SpiriSynq.schema*), 53

L

Link (*class in SpiriSynq.link*), 61
list_topics() (*SpiriSynq.session.Session method*), 51

M

make_generic_syncable_class() (*in module SpiriSynq.syncable_objects*), 61
module
    SpiriSynq.example_types.position, 62
    SpiriSynq.link, 61
    SpiriSynq.remote_callables, 53
    SpiriSynq.schema, 53
    SpiriSynq.session, 49
    SpiriSynq.syncable_objects, 56
mount (*SpiriSynq.example_types.position.Position attribute*), 68

N

ned_to_ecef() (*SpiriSynq.example_types.position.Orientation class method*), 64
normalize() (*SpiriSynq.example_types.position.Orientation method*), 64

O

objects (*SpiriSynq.session.Session attribute*), 50
Offset (*class in SpiriSynq.example_types.position*), 63
offset (*SpiriSynq.example_types.position.Pose attribute*), 66
Orientation (*class in SpiriSynq.example_types.position*), 63
orientation (*SpiriSynq.example_types.position.Pose attribute*), 66

P

payload (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
pitch (*SpiriSynq.example_types.position.Orientation property*), 64
Pose (*class in SpiriSynq.example_types.position*), 66
pose (*SpiriSynq.example_types.position.Position attribute*), 68
Position (*class in SpiriSynq.example_types.position*), 67
priority (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
PutArgs (*class in SpiriSynq.syncable_objects*), 56

R

register_codec() (*SpiriSynq.session.Session method*), 52
register_type_recursive() (*SpiriSynq.session.Session method*), 52
relative_to (*SpiriSynq.example_types.position.Position attribute*), 68
remote_method() (*in module SpiriSynq.remote_callables*), 56
RemoteMethod (*class in SpiriSynq.remote_callables*), 55
resolve() (*SpiriSynq.link.Link method*), 61
resolve_topic() (*SpiriSynq.session.Session method*), 51
retries() (*SpiriSynq.remote_callables.BoundRemoteMethod method*), 55
roll (*SpiriSynq.example_types.position.Orientation property*), 64
RootFrame (*class in SpiriSynq.example_types.position*), 62

rotate_by() (*SpiriSynq.example_types.position.Offset method*), 63
rpc_call() (*in module SpiriSynq.remote_callables*), 54
RpcException, 53

S

SCHEMA_VERSION (*in module SpiriSynq.schema*), 53
server() (*SpiriSynq.remote_callables.RemoteMethod method*), 55
Session (*class in SpiriSynq.session*), 49
setup_zenoh_callback() (*SpiriSynq.remote_callables.BoundRemoteMethod method*), 55
setup_zenoh_callback() (*SpiriSynq.remote_callables.RemoteMethod method*), 55
SKIP (*in module SpiriSynq.syncable_objects*), 56
source_info() (*SpiriSynq.session.Session method*), 52
SpiriSynq.example_types.position
    module, 62
SpiriSynq.link
    module, 61
SpiriSynq.remote_callables
    module, 53
SpiriSynq.schema
    module, 53
SpiriSynq.session
    module, 49
SpiriSynq.syncable_objects
    module, 56
sr_metadata() (*SpiriSynq.syncable_objects.SyncableObject method*), 60
sr_object_schema() (*SpiriSynq.syncable_objects.SyncableObject method*), 60
sr_rehydrate() (*SpiriSynq.syncable_objects.SyncableObject method*), 60
sr_rehydrate_client() (*SpiriSynq.syncable_objects.SyncableObject method*), 60
SubSyncableDataclass (*class in SpiriSynq.syncable_objects*), 56
sync() (*SpiriSynq.remote_callables.BoundRemoteMethod method*), 55
sync() (*SpiriSynq.syncable_objects.SyncableObject method*), 59
sync_dumps() (*SpiriSynq.syncable_objects.SyncableObject method*), 60
SyncableObject (*class in SpiriSynq.syncable_objects*), 57
SyncableObjectMetadata (*class in SpiriSynq.syncable_objects*), 56
synq_absolute_path (*SpiriSynq.syncable_objects.SyncableObject property*), 59
synq_authoritive (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_auto_rehydrate_on_missing_parent_timeout (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_auto_start (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_base_topic (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_check_receive_types (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_is_deleted (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_lazy_publish (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_mtime (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_publish (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_qos_for() (*SpiriSynq.syncable_objects.SyncableObject method*), 59
synq_receive (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_session (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_signal_missing_parent (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_signal_tombstone (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_signal_type_mismatch (*SpiriSynq.syncable_objects.SyncableObject*

attribute), 59
synq_signal_unknown_path (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_skip_rehydrate (*SpiriSynq.syncable_objects.SubSyncableDataclass attribute*), 57
synq_skip_rehydrate (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_skip_sync (*SpiriSynq.syncable_objects.SubSyncableDataclass attribute*), 57
synq_skip_sync (*SpiriSynq.syncable_objects.SyncableObject attribute*), 59
synq_topic (*SpiriSynq.syncable_objects.SyncableObject attribute*), 58
synq_type_tags() (*SpiriSynq.syncable_objects.SyncableObject class method*), 59

T

timeout() (*SpiriSynq.remote_callables.BoundRemoteMethod method*), 54
timestamp (*SpiriSynq.syncable_objects.PutArgs attribute*), 56
to() (*SpiriSynq.link.Link class method*), 61
to_cot() (*SpiriSynq.example_types.position.Accuracy method*), 66
to_euler() (*SpiriSynq.example_types.position.Orientation method*), 64
to_geodetic() (*SpiriSynq.example_types.position.Offset method*), 63
to_yaml() (*SpiriSynq.example_types.position.RootFrame class method*), 63
to_yaml() (*SpiriSynq.link.Link class method*), 62
to_yaml() (*SpiriSynq.syncable_objects.SubSyncableDataclass class method*), 57
to_yaml() (*SpiriSynq.syncable_objects.SyncableObject class method*), 60
topic (*SpiriSynq.link.Link attribute*), 61
topic (*SpiriSynq.syncable_objects.SyncableObjectMetadata attribute*), 56
type_registry (*SpiriSynq.session.Session attribute*), 50

U

undeclare() (*in module SpiriSynq.remote_callables*), 56

V

valid_sync_paths() (*SpiriSynq.syncable_objects.SubSyncableDataclass class method*), 57
valid_sync_paths() (*SpiriSynq.syncable_objects.SyncableObject class method*), 60
valid_sync_type() (*SpiriSynq.syncable_objects.SyncableObject class method*), 60
vertical (*SpiriSynq.example_types.position.Accuracy attribute*), 65

W

w (*SpiriSynq.example_types.position.Orientation attribute*), 64
WeakMethodProxy (*class in SpiriSynq.syncable_objects*), 57

X

x (*SpiriSynq.example_types.position.Offset attribute*), 63
x (*SpiriSynq.example_types.position.Orientation attribute*), 64

Y

y (*SpiriSynq.example_types.position.Offset attribute*), 63
y (*SpiriSynq.example_types.position.Orientation attribute*), 64
yaml_tag (*SpiriSynq.example_types.position.RootFrame attribute*), 62
yaml_tag (*SpiriSynq.link.Link attribute*), 61
yaml_tag() (*in module SpiriSynq.schema*), 53
yaw (*SpiriSynq.example_types.position.Orientation property*), 64

Z

z (*SpiriSynq.example_types.position.Offset attribute*), 63
z (*SpiriSynq.example_types.position.Orientation attribute*), 64
zenoh_session (*SpiriSynq.session.Session attribute*), 50